Decentralization

Q1. What is Decentralization? Explain its concept in the context of Blockchain Technology.

ANSWER

1. Introduction to Decentralization

Decentralization is not a new concept — it has been used in strategy, management, and government for a long time. The fundamental idea is to distribute control and authority to the peripheries of an organization rather than allowing a single central body to be in full control.

In the context of blockchain, decentralization means that no single central authority controls the network. This is the foundational principle of blockchain technology.

2. Decentralization Using Blockchain

Blockchain is designed as a perfect vehicle for providing a decentralized platform that:

- Does not require any intermediaries
- Can function with many different leaders chosen via consensus mechanisms
- Allows anyone to compete to become the decision-making authority
- Uses a consensus mechanism (most famously Proof of Work) to govern the process

Decentralization is applied in varying degrees, ranging from semi-decentralized to fully decentralized models, depending on requirements.

3. Traditional vs. Decentralized ICT Systems

Conventionally, Information and Communication Technology (ICT) has been based on a centralized paradigm where database or application servers are under the control of a central authority (like a system administrator).

With Bitcoin and blockchain, this model has fundamentally changed. Now the technology exists to:

- Allow anyone to start a decentralized system
- Operate with no single point of failure
- Operate with no single trusted authority
- Run either autonomously or with some human intervention depending on the governance model

4. Benefits of Decentralization in Blockchain

- Increased efficiency in operations
- Expedited decision-making through consensus
- Better motivation for participants
- Reduced burden on top management
- Transparency of all transactions
- Cost savings for consumers
- High availability and fault tolerance
- Collusion resistance through consensus algorithms

✍ **EXAM TIP:** Always mention that decentralization can remodel existing applications OR build new ones. Mention PoW as the most famous consensus mechanism. A 10-mark answer should cover definition, blockchain context, benefits, and briefly touch on challenges.

Q2. Differentiate between Centralized, Distributed, and Decentralized Systems with suitable diagrams/examples.

ANSWER

1. Centralized Systems

Centralized systems are conventional client-server IT systems in which:

- There is a single authority that controls the system
- That authority is solely in charge of all operations
- All users are dependent on a single source of service
- Examples: Google, Amazon, eBay, Apple's App Store

These represent the traditional model of computing where a single server or cluster under one owner provides all services to clients.

2. Distributed Systems

In a distributed system:

- Data and computation are spread across multiple nodes in the network
- Users view it as a single, coherent system despite the spread
- Variations are used to achieve fault tolerance and speed

Important distinction from parallel computing:

- Parallel computing: computation performed by all nodes simultaneously (e.g., weather forecasting, simulation, financial modeling)
- Distributed computing: computation may not happen in parallel; data is replicated across nodes

Key limitation: In the parallel system model (a form of distributed system), there is STILL a central authority that has control over all nodes and governs processing. This means the system is still centralized in nature.

3. Decentralized Systems

A decentralized system is a type of network where:

- Nodes are not dependent on a single master node
- Control is distributed among many nodes
- No central authority exists to govern the entire system
- Each participating node maintains an exact replica of applications and data

Analogy: Each department in an organization is in charge of its own database server — taking away power from the central server and distributing it to sub-departments.

4. Critical Difference: Distributed vs. Decentralized

This is the most important distinction for exam purposes:

Aspect	Distributed System	Decentralized System
Central Authority	Still exists — governs entire system	Does NOT exist

Control	Central controller manages all nodes	Control spread among all nodes
Example	Traditional web systems, cloud computing	Bitcoin, Ethereum blockchain
Data replication	May or may not replicate	Exact replicas on every node
Topology	Multiple servers, different roles	P2P connections, identical nodes

A decentralized system may look like a distributed system from a topological point of view, but the KEY difference is that it has NO central authority that controls the network.

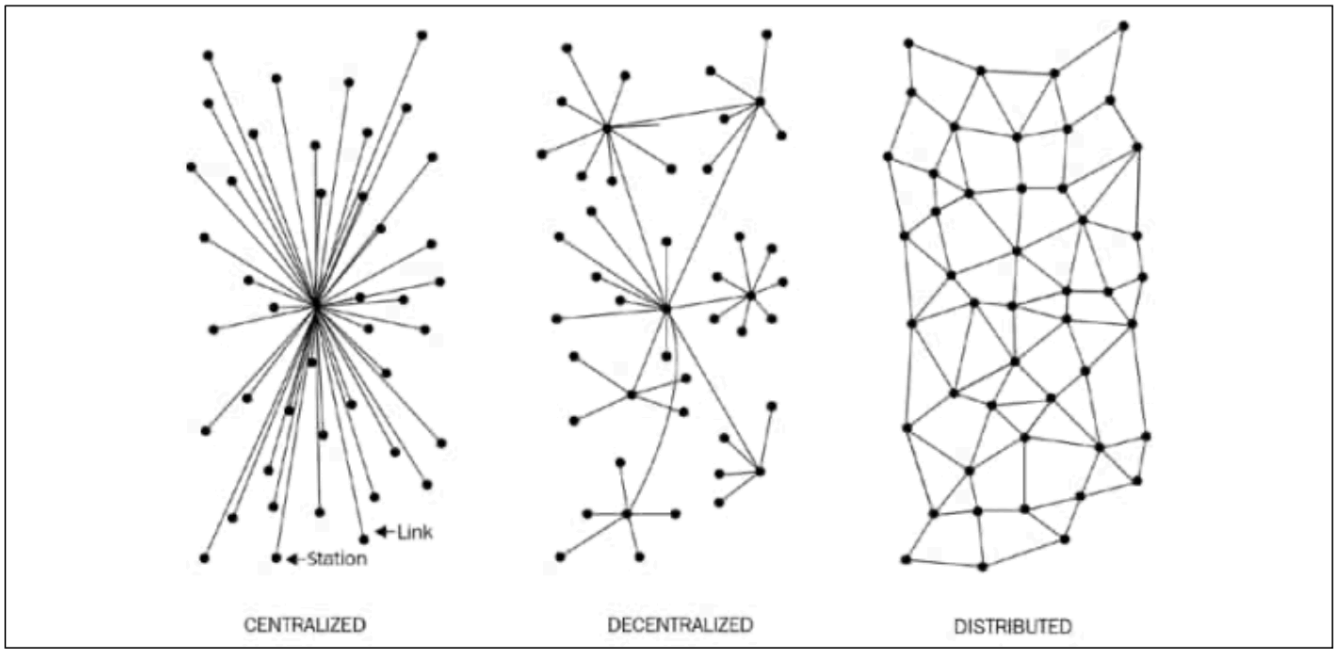

Figure 2.1: Different types of networks/systems

5. Summary Comparison Table

Feature	Centralized	Decentralized
Ownership	Service provider	All users
Architecture	Client/server	Distributed, different topologies
Security	Basic	More secure
High Availability	No	Yes
Fault Tolerance	Basic — single point of failure	Highly tolerant (service replicated)
Collusion Resistance	Basic (under control of group/individual)	High (consensus algorithms)
Application Architecture	Single application	Replicated across all nodes
Trust Requirement	Users must trust service provider	No mutual trust required
Cost for Consumer	Higher	Lower

✍ **EXAM TIP:** The most commonly asked trick question is 'Is a distributed system the same as a decentralized system?' — Always clarify: distributed systems still have a central authority; decentralized systems do not. Mentioning Paul Baran's 1964 work earns extra marks.

Q3. Explain the two methods of achieving Decentralization: Disintermediation and Contest-Driven Decentralization.

ANSWER

1. Overview

Two primary methods can be used to achieve decentralization:

- Disintermediation
- Contest-Driven (Competition-based) Decentralization

These methods differ in the degree of decentralization they achieve and in how they handle intermediaries.

2. Method 1: Disintermediation

> **Definition:** Disintermediation is the removal of intermediaries from a process or supply chain, allowing direct peer-to-peer interaction.

Classic Example — Cross-border Money Transfer:

- Traditional process: You → Bank (intermediary, charges fee) → Bank in foreign country → Recipient
- With blockchain: You → Recipient directly using blockchain address
- The bank (intermediary) is completely eliminated

This achieves FULL decentralization by completely removing the middleman.

Applications across industries:

- Finance: Direct peer-to-peer money transfers without banks
- Healthcare: Patients control their own health records — share directly with trusted entities (hospitals, pharma companies) without a central record system
- Law: Smart contracts executing automatically without lawyers as intermediaries
- Public Sector: Citizens directly accessing government services without bureaucratic intermediaries

Challenges of Disintermediation:

- In the financial sector, massive regulatory and compliance requirements make it difficult
- Legal frameworks may not recognize blockchain-based transactions in all jurisdictions

3. Method 2: Contest-Driven Decentralization

> **Definition:** In this method, different service providers compete with each other to be selected for the provision of services by the system.

Key characteristics:

- Does NOT achieve complete decentralization
- Ensures no single intermediary monopolizes the service
- Uses reputation, previous scores, reviews, and quality of service as selection criteria

Blockchain Application Example:

- Smart contracts can choose an external data provider (oracle) from a large number of providers
- Selection based on: reputation, previous score, reviews, quality of service
- This cultivates an environment of competition among service providers

4. Scale of Decentralization

Decentralization exists on a spectrum from fully centralized to fully decentralized:

Level	Description	Example
Fully Centralized	Central intermediaries are in control	Current financial system (banks)
Semi-Decentralized	Intermediaries compete with each other; selected by reputation/voting	Multiple service providers competing for smart contract
Fully Decentralized	No intermediaries required	Bitcoin — direct P2P transactions

5. Comparison of Both Methods

Aspect	Disintermediation	Contest-Driven
Degree of Decentralization	Full (complete removal)	Partial (intermediary still exists)
Intermediary Role	Completely removed	Competes with others; selected by merit
Best For	Finance, healthcare, supply chains	Oracle selection, data providers
Example	Bitcoin P2P transfer	Multiple oracles competing for smart contract selection

✍ **EXAM TIP:** Always give the bank/money transfer example for disintermediation — it is the textbook example. For contest-driven, mention oracle/data provider selection by smart contracts. Earn extra marks by mentioning the scale: Fully Centralized → Semi-decentralized → Fully Decentralized.

Q4. What is Decentralized Consensus? Why is it considered a significant innovation? Explain its role in blockchain.

ANSWER

1. Definition

Decentralized Consensus: A mechanism that enables users to agree on something (reach consensus) via a consensus algorithm without the need for a central, trusted third party, intermediary, or service provider.

This mechanism first came into play with Bitcoin and represents one of the most significant innovations in the history of computing and distributed systems.

2. Why It Is a Significant Innovation

Before decentralized consensus, any system requiring agreement between multiple parties needed:

- A trusted central authority (e.g., a bank, government, notary)
- An intermediary to verify and record the state
- Users to blindly trust the central party

Decentralized consensus eliminated all of these requirements. It gave rise to a new era of decentralized applications where:

- No single entity needs to be trusted
- Agreement is achieved algorithmically
- The system is transparent and verifiable by all participants
- Resistance to manipulation is built into the protocol

3. How It Works in Blockchain

In a blockchain context, decentralized consensus works by:

- Multiple nodes independently verifying transactions
- A consensus algorithm determining which version of the truth is accepted
- The most famous method being Proof of Work (PoW) — where nodes compete to solve a computational puzzle to add the next block
- The 'winning' node broadcasts the block; others verify and accept it if valid

This model allows anyone to compete to become the decision-making authority in a given round without any permanent central controller.

4. Relationship to Decentralization

Decentralized consensus is the technical enabler of blockchain decentralization:

- Without consensus, multiple nodes would have conflicting versions of the ledger
- Consensus ensures all honest nodes agree on the same state
- It replaces the need for a bank, notary, or other trusted third party

This is why decentralized consensus is described as the 'significant innovation in the decentralized paradigm that has given rise to this new era of decentralization of applications.'

✍ **EXAM TIP:** Link decentralized consensus to Proof of Work (PoW) and mention that it removes the need for a trusted third party. If asked in the context of blockchain, also mention that this is what enables Bitcoin and Ethereum to function without banks or central servers.

Q5. When should Blockchain be used? Discuss the decision framework for choosing blockchain vs. traditional databases.

ANSWER

1. The Core Question

A fundamental question that must be asked before adopting blockchain is: Is a blockchain really needed? Before embarking on a journey to decentralize everything, it is essential to understand that not everything can or needs to be decentralized.

2. Decision Framework

The following set of questions helps determine whether blockchain is appropriate:

Question	Yes → Recommended	No → Recommended
Is high data throughput required?	Use a traditional database	May still use central database if trust exists; blockchain if trust is absent

Are updates centrally controlled?	Use a traditional database	Investigate public/private blockchain
Do users trust each other?	Use a traditional database	Use a public blockchain
Are users anonymous?	Use a public blockchain	Use a private blockchain
Is consensus required within a consortium?	Use a private blockchain	Use a public blockchain
Is strict data immutability required?	Use a blockchain	Use a central/traditional database

3. When Traditional Database Is Better

- High data throughput applications (blockchain is slower due to consensus overhead)
- Updates are controlled by a single trusted authority
- All users already trust each other
- No immutability requirement
- Real-time read/write operations are needed

4. When Blockchain Is Better

- No mutual trust exists between participants
- Immutability of records is critical
- Decentralized governance is required
- Users are anonymous and cannot be identified
- Consortium of organizations need shared, trusted ledger
- Eliminating a central intermediary provides significant value

5. Types of Blockchain Based on Requirements

- Public Blockchain: When users don't trust each other and anonymity is needed (e.g., Bitcoin, Ethereum)
- Private Blockchain: When a consortium needs consensus but users are not anonymous (e.g., Hyperledger)

✍ **EXAM TIP:** The decision table is a frequent 10-mark question. Memorize the six criteria and their outcomes. A common wrong answer is 'always use blockchain' — the correct answer is context-dependent. Mentioning 'data throughput' and 'immutability' as key deciding factors earns extra marks.

Q6. Discuss the Benefits and Challenges of Decentralization in Blockchain.

ANSWER

1. Benefits of Decentralization

A. Transparency

- All transactions on a public blockchain are visible to all participants
- The entire transaction history is publicly auditable
- No hidden modifications can be made by any single party

B. Efficiency

- Removal of intermediaries speeds up processes

- Smart contracts automate execution without manual intervention
- Cross-border transactions happen in minutes rather than days

C. Cost Saving

- No intermediary fees (e.g., no bank fees for international transfers)
- Lower costs for end consumers
- Reduced operational overhead for businesses

D. Development of Trusted Ecosystems

- Trust is established through cryptography and consensus — not through reputation of a third party
- No mutual trust required between users
- Smart contracts enforce agreements automatically

E. Privacy and Anonymity (in some cases)

- Public blockchains allow pseudonymous participation
- Users can interact without revealing personal identity

F. High Availability and Fault Tolerance

- Service is replicated across all nodes
- No single point of failure
- Network continues functioning even if some nodes go offline

G. Collusion Resistance

- Consensus algorithms ensure defense against adversaries
- Highly resistant to manipulation — changing data requires controlling majority of network

2. Challenges of Decentralization

A. Security Requirements

- In systems like Bitcoin/Ethereum, security relies on private keys
- Assets associated with lost or stolen private keys cannot be recovered
- If private keys are lost due to user negligence, assets become permanently inaccessible

B. Software Bugs

- Smart contract code, once deployed, is immutable
- Bugs in smart contract code can make decentralized applications vulnerable to attack
- Historical example: The DAO hack on Ethereum caused by a vulnerability in smart contract code

C. Human Error

- Unlike centralized systems, there is no administrator to correct mistakes
- Wrong transactions cannot be reversed once confirmed
- Sending to a wrong address results in permanent, unrecoverable loss

D. Scalability

- Decentralized systems generally have lower throughput than centralized databases
- Consensus mechanisms add overhead, reducing speed

E. Regulatory Challenges

- Disintermediation in finance is challenging due to massive regulatory and compliance requirements
- Legal frameworks may not recognize decentralized transactions

3. Fundamental Questions to Ask

- Is a blockchain really needed?
- Can trust be established through other means?
- What is the risk if private keys are lost?
- Is the system resilient to smart contract bugs?

✍ **EXAM TIP:** Structure your answer under 'Benefits' and 'Challenges' with subheadings. For a 10-mark answer, aim for at least 4–5 benefits and 3–4 challenges, each briefly elaborated. The private key security issue and smart contract bug risk are specific to blockchain — always include them.

Q7. What are Decentralized Applications (DApps)? How do they differ from traditional applications?

ANSWER

1. Definition of DApps

Decentralized Application (DApp): An application that runs on a decentralized blockchain network where the application's code and data are replicated across all participating nodes, with no single central authority controlling it.

2. How DApps Work

- The application logic is encoded in smart contracts deployed on the blockchain
- Each participating node runs the same application code
- Execution results are verified by consensus among the nodes
- The state is stored on the blockchain — immutable and transparent
- Users interact with the DApp through a frontend (web/mobile) that communicates with the blockchain

3. Key Characteristics of DApps

- Open Source: Code is typically open source and changes require majority consensus
- Decentralized: Data stored on a decentralized blockchain
- Incentivized: Validators/miners are rewarded with tokens for contributing resources
- Protocol-based: Uses a consensus mechanism for validation
- No Downtime: Application runs as long as at least one node is operational

4. DApps vs. Traditional Applications

Feature	Traditional Application	DApp
Hosting	Central server (owned by company)	Distributed across all blockchain nodes
Data Control	Service provider controls data	All users collectively control data
Availability	Depends on central server uptime	No single point of failure
Transparency	Code is proprietary/hidden	Code is open source on blockchain
Trust	Users must trust the provider	Trust is built into the protocol
Cost	Provider charges for service	Users pay gas/transaction fees directly
Fault Tolerance	Single point of failure	Highly fault tolerant
Modification	Provider can unilaterally update	Changes require consensus

5. Governance of DApps

DApps can either be:

- Run autonomously (fully automated smart contract logic)
- Require some human intervention, depending on the type and model of governance used

✍ **EXAM TIP:** Emphasize 3 key points: (1) No single point of failure, (2) No single trusted authority needed, (3) Application is replicated across all nodes. These are direct quotes from the textbook and earn full marks.

QUICK REFERENCE: Key Terms & One-Line Definitions

Term	Definition
Decentralization	Distribution of control and authority to the peripheries, eliminating a single central controlling body
Centralized System	Conventional client-server system with a single authority in full control
Distributed System	Data/computation spread across multiple nodes but still with a central governing authority
Decentralized System	Network where no single master node exists; control distributed among all nodes
Disintermediation	Removal of intermediaries to enable direct peer-to-peer interaction
Contest-Driven Decentralization	Service providers compete based on reputation/score to be selected — partial decentralization
Decentralized Consensus	Mechanism to agree on something without a trusted third party using consensus algorithms
Proof of Work (PoW)	Most famous consensus mechanism — nodes compete to solve a puzzle to add the next block
DApp	Decentralized Application — runs on a blockchain with code replicated across all nodes
Smart Contract	Self-executing code stored on blockchain that enforces agreement automatically
Public Blockchain	Open blockchain for anonymous users who don't trust each other (e.g., Bitcoin, Ethereum)
Private Blockchain	Permissioned blockchain for known users within a consortium
Fault Tolerance	Ability of system to continue operating even when some components fail
Immutability	Property of blockchain data that makes it tamper-proof once recorded

MUST-KNOW POINTS FOR EXAM

1. Distributed ≠ Decentralized. Distributed systems still have a central authority. Decentralized systems do NOT.
2. Two methods of decentralization: Disintermediation (full — removes intermediary) and Contest-driven (partial — intermediaries compete).
3. Not everything needs to be decentralized — use the 6-question decision framework to determine when blockchain is appropriate.
4. High data throughput → Traditional database. No mutual trust + immutability required → Blockchain.
5. The concept of different network types (centralized/distributed/decentralized) was first published by Paul Baran in 1964.
6. In a blockchain-based decentralized system, every node holds an EXACT replica of all blocks (unlike distributed systems where different servers have different roles).

Q1. What are the Routes to Decentralization? Explain the Narayanan Framework for evaluating decentralization with a suitable example.

ANSWER

1. Routes to Decentralization - Background

Before blockchain and Bitcoin, systems like BitTorrent and Gnutella file-sharing could be classified as partially decentralized. However, they lacked an incentivization mechanism - no incentive for users to keep participating - so community participation gradually decreased.

With blockchain, two primary routes now exist:

- Bitcoin blockchain - first choice for many; most resilient and secure blockchain
- Ethereum - preferred by developers for building DApps due to flexibility of smart contracts

2. The Narayanan Framework (4 Questions)

Arvind Narayanan et al. proposed a framework in 'Bitcoin and Cryptocurrency Technologies' to evaluate decentralization requirements of any system. It raises four questions:

#	Question	Purpose
1	What is being decentralized?	Identify the system - identity system, trading system, money transfer, etc.
2	What level of decentralization is required?	Determine the scale - full disintermediation or partial disintermediation
3	What blockchain is used?	Choose the appropriate blockchain - Bitcoin, Ethereum, or other suitable chain
4	What security mechanism is used?	Specify how security will be guaranteed - atomicity-based, reputation-based, etc.

3. Framework Applied - Money Transfer System Example

Question	Answer
What is being decentralized?	Money transfer system
What level of decentralization?	Disintermediation - complete removal of the bank (intermediary)

What blockchain is used?	Bitcoin - longest established blockchain; has stood the test of time
What security mechanism?	Atomicity - transaction executes fully or not at all (deterministic integrity)

The responses indicate: Remove the bank entirely, implement on Bitcoin, and guarantee security via atomicity - ensuring transactions either succeed completely or fail completely.

4. Security Mechanisms

Atomicity-based: Transaction executes fully or not at all - deterministic approach ensuring integrity of the system

Reputation-based: Allows varying degrees of trust; participants selected based on established reputation or track record

5. Why This Framework Matters

- Provides a structured approach applicable to ANY system needing decentralization
- Helps developers choose the right blockchain and security model
- Prevents over-engineering - ensures decentralization is applied only where appropriate
- Pre-blockchain systems (BitTorrent, Gnutella) lacked incentivization; blockchain solves this

EXAM TIP: Always present all 4 questions as a numbered framework. The money transfer -> Bitcoin -> Atomicity example is the textbook answer. Define atomicity clearly: 'transaction executes fully or not at all'. Mention Bitcoin was chosen because it is the longest established blockchain and has stood the test of time.

Q2. Explain Full Ecosystem Decentralization. Discuss the layers of a decentralized ecosystem including Storage, Communication, and Computing Power.

ANSWER

1. Overview

To achieve complete decentralization, the environment around the blockchain must also be decentralized. The blockchain is a distributed ledger that runs on top of conventional systems. Three core elements must be decentralized:

- Storage
- Communication
- Computation (Computing Power)

Additionally, identity and wealth - traditionally centralized - must also be decentralized for a sufficiently decentralized ecosystem.

2. Layer 1 - Storage Decentralization

A. Direct Blockchain Storage

- Data can be stored directly in a blockchain - achieves decentralization
- Significant disadvantage: NOT suitable for large amounts of data by design
- Suitable for: simple transactions and small arbitrary data
- NOT suitable for: images, large data blobs (unlike traditional databases)

B. Distributed Hash Tables (DHTs)

A better alternative for large data storage. Used in peer-to-peer file sharing software (BitTorrent, Napster, Kazaa, Gnutella). Research popularized by CAN, Chord, Pastry, and Tapestry projects.

- BitTorrent is most scalable and fastest - but lacks permanent incentive mechanism
- Problem: users don't keep files permanently; if a node leaves, files become unavailable
- Two primary requirements: high availability and link stability

C. IPFS - InterPlanetary File System

IPFS: Created by Juan Benet. Provides a decentralized World Wide Web by replacing HTTP. Possesses both high availability and link stability.

- Uses Kademlia DHT for storage functionality
- Uses Merkle Directed Acyclic Graphs (DAGs) for searching functionality
- Incentive mechanism: Filecoin protocol - pays nodes to store data using the Bitswap mechanism
- Bitswap: nodes keep a ledger of bytes sent vs. bytes received in a 1-to-1 relationship
- Uses Git-based version control for structure and versioning of data

D. Other Storage Alternatives

- Ethereum Swarm - decentralized storage for Ethereum ecosystem
- Storj - decentralized cloud storage
- MaidSafe - aims to provide a decentralized World Wide Web
- BigChainDB - fast, linearly scalable decentralized database; complements IPFS and Ethereum

3. Layer 2 - Communication Decentralization

The Internet is considered decentralized, but this is only partially true. The original vision was decentralized; however, services (email, storage) are now controlled by central providers. Users are NOT in control of their data - even passwords stored on third-party systems. ISPs act as central hubs - if ISP shuts down, communication fails.

Decentralizing Communication

- Mesh networks: nodes communicate directly without a central hub (ISP)
- Example: Firechat - allows iPhone users to communicate P2P without Internet connection
- Whisper protocol - decentralized communication protocol in Ethereum ecosystem

Note on email: Email is fundamentally decentralized (anyone can run an email server), but Gmail and Outlook became dominant centralized alternatives due to convenience - showing how the Internet drifted toward centralization. Free services come at the cost of exposing valuable personal data.

4. Layer 3 - Computing Power Decentralization

Achieved by blockchain technology such as Ethereum, where smart contracts with embedded business logic run on the blockchain network. Other blockchain technologies provide similar processing-layer platforms where business logic runs over the network in a decentralized manner.

5. The Decentralized Ecosystem - Layer Model

Layer (Bottom to Top)	Technologies
Communication (Base Layer)	The Internet, Mesh networks, Whisper protocol
Storage	Filesystems: IPFS, Swarm, Storj | Database: BigChainDB
Blockchain (Processing/Computation)	Bitcoin, Ethereum, EOS, Tezos
Decentralized Identity, Finance, Wealth & Web (Top Layer)	bitAuth, OpenID, Namecoin, DeFi platforms

6. Zooko's Triangle - The Identity Challenge

Zooko's Triangle: A conjecture stating that a naming system in a network protocol can possess at most TWO of the following three properties simultaneously: (1) Secure, (2) Decentralized, (3) Human-meaningful names.

Resolution: With the Namecoin blockchain, all three properties can be achieved simultaneously. However, challenges remain - users must store and maintain private keys securely.

EXAM TIP: Always present the ecosystem as a layered diagram/table: Communication (bottom) -> Storage -> Blockchain -> Identity/Wealth (top). For IPFS mention BOTH components: Kademlia DHT (storage) + Merkle DAG (searching). Filecoin is the incentive layer. Zooko's Triangle (secure + decentralized + human-meaningful) and Namecoin as its solution earn extra marks.

Q3. Explain the key terminology in Decentralization: Smart Contracts, Autonomous Agents (AA), Decentralized Organizations (DO), DAOs, DACs, and DASes. Compare them using a table.

ANSWER

1. Smart Contracts

Smart Contract: A software program that usually runs on a blockchain. Contains business logic and a limited amount of data. The business logic executes if specific criteria are met.

- Does not necessarily need a blockchain to run
- Blockchain has become the standard decentralized execution platform due to security benefits
- Actors/participants use smart contracts, or they run autonomously on behalf of network participants
- In a DAO, the code (smart contract) is the governing entity - not people or paper contracts

2. Autonomous Agents (AA)

Autonomous Agent: An artificially intelligent software entity that acts on behalf of its owner to achieve desirable goals without requiring any or minimal intervention from its owner.

- Fully automated - operates without human intervention
- Could be used by law enforcement/regulators to embed compliance rules into a DAO

3. Decentralized Organizations (DO)

DO: A software program that runs on a blockchain based on the idea of actual organizations with people and protocols. Once added as a smart contract, it becomes decentralized.

- Parties interact based on code defined within the DO software
- NOT autonomous - relies on human input to execute business logic
- Has ownership structure and established legal status

4. Decentralized Autonomous Organizations (DAO)

DAO: A computer program running on a blockchain with embedded governance and business logic rules. Same as a DO but fully autonomous - contains AI logic and does NOT rely on human input.

Key characteristics:

- Ethereum blockchain led the way with DAOs

- The code is the governing entity - not people or paper contracts
- A human curator maintains the code and acts as a proposal evaluator
- Can hire external contractors if enough input received from token holders
- Currently have NO legal status
- Can run in any jurisdiction - purely decentralized entities

The DAO - Famous Case Study

- The most famous DAO project raised $168 million in crowdfunding
- Designed as a venture capital fund with no single entity as owner
- Hacked due to a bug in the code - millions of ETH siphoned into a child DAO by hackers
- Required a major hard fork on Ethereum to reverse the hack and recover funds
- Raised critical debate on security, code quality, and thorough testing of smart contracts

5. Decentralized Autonomous Corporation (DAC)

DAC: Similar to a DAO in concept; considered a subset. Can earn profit via shares offered to participants and can pay dividends. Runs a business automatically without human intervention.

- DAOs are generally considered non-profit
- DACs can earn profit via shares and pay dividends

6. Decentralized Autonomous Society (DAS)

DAS: A concept whereby an entire society can function on a blockchain using multiple complex smart contracts and a combination of DAOs and DApps running autonomously.

- Not a free-for-all - government services can be delivered via blockchains
- Examples: government identity cards, passports, records of deeds, marriages, births
- If a government is corrupt, society can start its own virtual one driven by decentralized consensus

7. Comparison Table - All Decentralized Entities

Entity	Autonomous?	Software?	Owned?	Capital?	Legal Status	Cost
DO	No	No	Yes	Yes	Yes (established)	High
DAO	Yes	Yes	No	Yes	Unsettled	Low
DAC	Yes	Yes	Yes	Yes	Unsettled	Low
DAS	Yes	Yes	No	Possible	Unsettled	Low
DApp	Yes	Yes	Yes	Optional tokens	Unsettled	Use-case dependent

EXAM TIP: The comparison table is frequently asked directly in exams - memorize all 6 columns for all 5 entities. Key distinctions: (1) DO is NOT autonomous - it relies on human input; (2) DAO IS autonomous and non-profit; (3) DAC is autonomous and CAN earn profit; (4) Only DO has established legal status; (5) The DAO hack ($168M) is the most important real-world example - always include it.

Q4. What are Decentralized Applications (DApps)? Explain their Types, Requirements, Architecture, and give examples.

ANSWER

1. Definition

DApp: A software application that runs on a decentralized network such as a distributed ledger. DAOs, DACs, and DOs are all DApps running on a blockchain in a peer-to-peer network.

DApps represent the latest advancement in decentralization technology. Thousands of DApps now run on various platforms covering media, social, finance, games, insurance, and health. The highest number run on Ethereum.

2. Types of DApps

Type 1 DApps

- Run on their own dedicated blockchain
- Make use of a native token (e.g., ETH on Ethereum)
- Example: Standard smart contract-based DApps on Ethereum

Type 2 DApps

- Use an existing established (Type 1) blockchain
- Bear custom protocols and tokens
- Example: DAI - built on Ethereum but has its own stablecoins and distribution mechanism
- Example: Golem - own token GNT; decentralized marketplace for computing power built on Ethereum
- Example: OMNI network - software layer on top of Bitcoin for trading custom digital assets

Type 3 DApps

- Use the protocols of Type 2 DApps
- Example: SAFE Network uses the OMNI protocol (a Type 2 DApp)
- Example: USDT (Tether) - uses OMNI layer (Type 2) on Bitcoin (Type 1) making it Type 3

NOTE: In recent years, the clear distinction between DApp types is less commonly referenced. DApps are now generally considered as apps running on blockchains like Ethereum, Tezos, NEO, or EOS without specific type reference.

3. Requirements of a DApp (Johnston et al., 2015)

From 'The General Theory of Decentralized Applications, DApps' - four criteria must be met:

- Open Source and Autonomous: Fully open source; no single entity controls majority of tokens; all changes must be consensus-driven based on community feedback
- Cryptographically Secured: Data and records stored on a public, decentralized blockchain; cryptographically secured to avoid central points of failure
- Cryptographic Token: Must use a cryptographic token to provide access and incentivize contributors (e.g., miners in Bitcoin)
- Token Generation by Consensus: Tokens generated by the DApp using consensus and a cryptographic algorithm - acts as proof of value to contributors

4. Operations of a DApp

- Consensus via PoW (Proof of Work) or PoS (Proof of Stake)
- PoW has proven incredibly resistant to attacks - evident from Bitcoin's success
- Tokens/coins distributed via: mining, fundraising, or development

5. Architecture Comparison

Component	Traditional App (Client/Server)	DApp
Backend	Database server (centralized)	Blockchain running smart contracts
Logic Layer	Web/Application server	Smart contracts (embedded business logic)
Frontend	Web UI / Mobile app	Web UI / Mobile (React, Angular, etc.)
Data Storage	Centralized proprietary database	Decentralized blockchain
Access Method	Via HTTP to web server	Via API to blockchain network
Control	Single owner/company	Distributed among all users
Trust	Must trust service provider	Trust is in the protocol/code

Key element in DApp creation: A smart contract that runs on the blockchain with business logic embedded within it. Frontend can be thick client, mobile app, or web frontend (usually JavaScript framework - React or Angular).

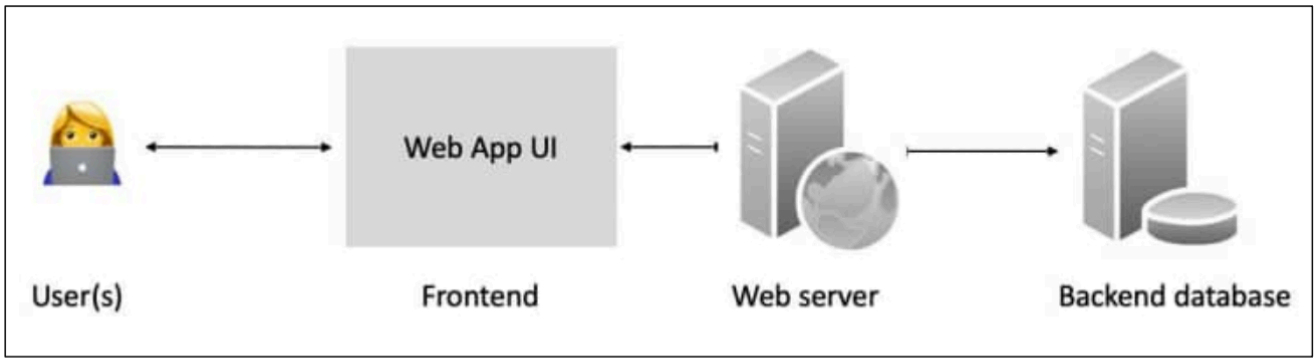

Figure 2.7: Traditional application architecture (generic client/server)

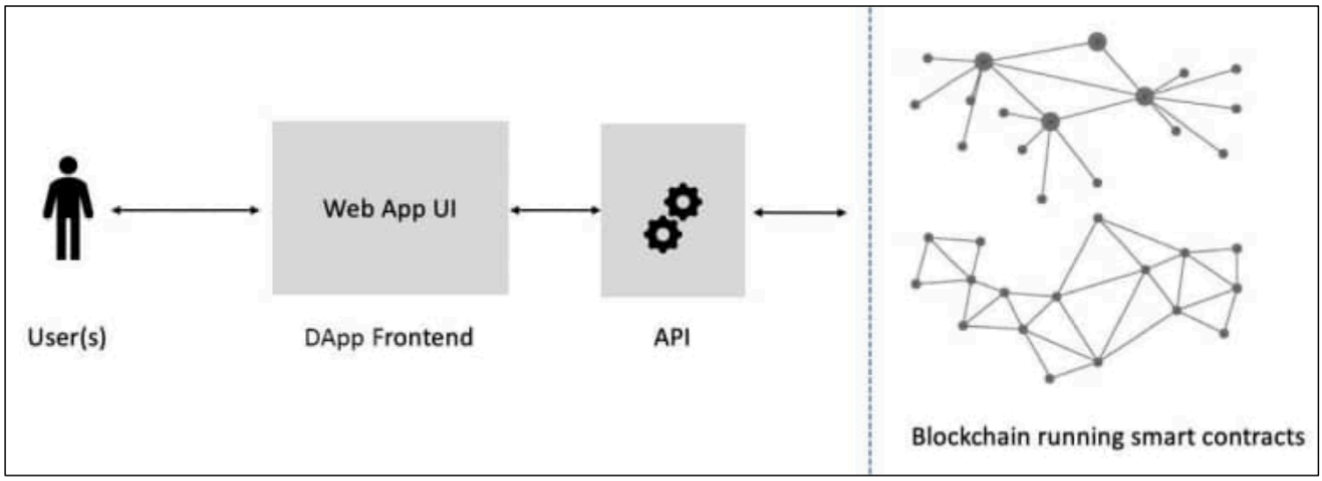

Figure 2.8: Generic DApp architecture

The key element that plays a vital role in the creation of a DApp is a smart contract that runs on the blockchain and has business logic embedded within it: Note that the frontend in either a DApp or app architecture can either be a thick client, a mobile app, or a web frontend (a web user interface). However, it is usually a web frontend commonly written using a JavaScript framework such as React or Angular.

6. DApp Examples

KYC-Chain: Manages Know Your Customer (KYC) data securely using smart contracts

OpenBazaar: Decentralized P2P network for direct commerce between buyers and sellers; uses Bitcoin; built on DHTs not blockchain

Lazooz: Decentralized equivalent of Uber; P2P ride sharing; users earn Zooz coins via proof of movement

EXAM TIP: For DApp types, USDT/Tether is the perfect Type 3 example. For requirements, present all 4 points from Johnston et al. For architecture, contrast Traditional (User -> Frontend -> Web Server -> DB) with DApp (User -> Frontend -> API -> Blockchain + Smart Contracts). Mention that only PoW has so far proven incredibly resistant to attacks.

Q5. Describe the major Platforms for Decentralization. Compare their features and use cases.

ANSWER

1. Introduction

Any blockchain network - Bitcoin, Ethereum, Hyperledger Fabric, or Quorum - can provide a decentralization service. Many organizations have introduced platforms making distributed application development easy, accessible, and secure.

2. Ethereum

- First blockchain to introduce a Turing-complete programming language (Solidity)
- First to introduce the concept of a Virtual Machine (EVM - Ethereum Virtual Machine)
- First proposed in 2013 by Vitalik Buterin
- Provides a public blockchain for smart contracts and DApps
- Currency: Ether (ETH)
- Stark contrast to Bitcoin's limited scripting language - Solidity enables endless DApp possibilities
- Highest number of DApps currently run on Ethereum

3. MaidSafe

- Project for decentralized Internet introduced in 2006
- NOT a blockchain - a decentralized and autonomous network
- Provides SAFE (Secure Access for Everyone) network
- Made up of unused computing resources (storage, processing, data connections) from users
- Files divided into small encrypted chunks distributed randomly across the network
- Data can only be retrieved by its respective owner
- Key innovation: Duplicate files automatically rejected - reduces computing resources needed
- Currency: Safecoin (incentivizes contributors)

4. Lisk

- Blockchain application development and cryptocurrency platform
- Developers use JavaScript to build DApps hosted in sidechains
- Consensus: Delegated Proof of Stake (DPOS) - 101 nodes elected to secure network and propose blocks
- Backend: Node.js and JavaScript | Frontend: CSS3, HTML5, JavaScript
- Currency: LSK coin
- Derivative: Rise - Lisk-based DApp platform with greater security focus

5. EOS

- Blockchain protocol launched January 2018; cryptocurrency: EOS
- Raised $4 billion USD in 2018 through Initial Coin Offering (ICO)
- Key purpose: Build a decentralized operating system
- Throughput comparison:
  - ~ EOS: approximately 3,996 transactions per second (TPS)
  - ~ Ethereum: approximately 15 TPS
  - ~ Bitcoin: approximately 7 TPS

6. Platform Comparison Table

Platform	Language	Consensus	Key Feature	Token
Ethereum	Solidity (Turing-complete)	PoW (transitioning to PoS)	First smart contract platform; largest DApp ecosystem	ETH
MaidSafe	Multiple	Custom autonomous	NOT a blockchain; SAFE network; encrypted chunks; rejects duplicates	Safecoin
Lisk	JavaScript	DPOS (101 nodes)	Sidechains; JS-based DApp development for mainstream developers	LSK
EOS	C++	DPOS	Highest throughput (~3,996 TPS); vision of decentralized OS; $4B ICO	EOS
Bitcoin	Script (limited)	PoW	Most secure and resilient; ideal for value transfer; longest track record	BTC

EXAM TIP: Always include TPS numbers: EOS (~3,996) >> Ethereum (~15) >> Bitcoin (~7). These exact figures are highly valued in exams. For Ethereum, stress it was FIRST to introduce Turing-complete language and EVM. For MaidSafe, clearly state it is NOT a blockchain - this distinction is commonly tested. EOS's $4B ICO is a memorable fact.

Q6. Explain the evolution of the Web - Web 1, Web 2, and Web 3. How does blockchain enable the Decentralized Web (Web 3)?

ANSWER

1. The Concept of Decentralized Web

> **Decentralized Web (Web 3):** A vision of the web where no central authority or set of authorities will be in control, restoring the original decentralized intention of the Internet.

The original intention of the Internet was decentralized. Open protocols (HTTP, SMTP, DNS) meant anyone could use them freely and become part of the Internet. However, large profit-seeking companies (Facebook, Google, Twitter, Amazon) shifted control toward themselves, creating a centralized and closed system - raising major concerns around privacy and data protection.

2. Web 1 - The Static Web

- Original World Wide Web developed in 1989
- Static web pages hosted on servers

- Users could only READ - no interactivity or user-generated content
- Read-only experience

3. Web 2 - The Interactive/Service Web (Current Era)

- Era of service-oriented and web-hosted applications
- Features: e-commerce, social networking, social media, blogs, multimedia sharing, web apps
- Richer and more interactive Internet experience
- ALL services are still CENTRALIZED
- Generated massive economic value; essential for business and personal life
- Problems: privacy concerns, need for trusted third parties, data breaches
- Examples: Twitter, Facebook, Google Docs, Gmail, Hotmail

Web 2 provides rich services but at the cost of centralization, privacy violations, and dependence on a handful of powerful companies. Free services are offered at the cost of exposing valuable personal data - many users are unaware of this fact.

4. Web 3 - The Decentralized Web (Future)

- Vision of a fully user-centric and decentralized internet
- No single authority or large organization in control
- Restored version of the Internet's original decentralized vision
- Blockchain is the enabling technology for Web 3

Examples of Web 3 applications:

Steemit: Social media platform on Steem blockchain; rewards contributors with STEEM cryptocurrency for content - more votes = more tokens

Status: Decentralized multipurpose communication platform providing secure and private communication

IPFS: Peer-to-peer hypermedia/storage protocol for decentralized storage and sharing across a P2P network

5. Comparison: Web 1 vs Web 2 vs Web 3

Feature	Web 1 (1989+)	Web 2 (2000s-Now)	Web 3 (Emerging)
Content	Static pages	Dynamic, interactive	Decentralized, user-owned
User Role	Read-only	Read + Write	Read + Write + Own
Control	Server owners	Large corporations (Google, Facebook)	Users (via blockchain)
Data Ownership	Server owner	Platform (sells user data)	Individual users
Technology	HTML, basic HTTP	AJAX, APIs, Cloud services	Blockchain, Smart contracts, IPFS
Privacy	Moderate	Low (data monetized)	High (cryptographic protection)
Trust	Trust the server owner	Trust the platform	Trust the protocol/code
Examples	Early websites, directories	Twitter, Facebook, Gmail	Steemit, Status, IPFS, DeFi apps

EXAM TIP: The Web 1 to 2 to 3 evolution is a very common 10-mark question. The key formula: Web 1 = Read-only | Web 2 = Read+Write (Centralized) | Web 3 = Read+Write+OWN (Decentralized). Always state

that blockchain is the enabling technology for Web 3. Include examples for each era, and mention that free Web 2 services come at the cost of exposing personal data.

Q7. What is Decentralized Identity? Explain its importance, Zooko's Triangle, and relevant initiatives.

ANSWER

1. The Identity Problem

Identity is a sensitive and difficult problem. Currently, due to the dominance of large Internet companies, the identity of a user is NOT in control of the identity holder - often leading to privacy issues.

- Users depend on third-party platforms (Google, Facebook) for authentication
- These platforms control when and how identity is used
- Identity breaches and data theft are significant risks

2. What is Decentralized Identity?

Decentralized Identity: A model that gives control of identity credentials back to identity holders, enabling them to control when and how they share their credentials and with whom.

- No single central authority controls the identity
- Users selectively disclose identity attributes
- Based on cryptographic proofs rather than trust in a third party

3. Key Initiatives

- Microsoft ION (Identity Overlay Network): Built on top of Bitcoin blockchain; based on W3C work and the Decentralized Identity Foundation
- IBM: Has taken similar decentralized identity initiatives
- W3C DIDs (Decentralized Identifiers): Standard specification for decentralized identity
- Decentralized Identity Foundation (DIF): Organization driving open standards for decentralized identity

4. Zooko's Triangle

Zooko's Triangle: A conjecture requiring that a naming system in a network protocol be: (1) Secure, (2) Decentralized, and (3) Human-meaningful. The conjecture states a system can have only TWO of these three properties simultaneously.

Property	Meaning
Secure	Names cannot be forged, stolen, or hijacked
Decentralized	No central authority required to resolve or manage names
Human-meaningful	Names are memorable and understandable by humans (not cryptographic hashes)

Resolution via Namecoin blockchain: It is now possible to achieve all three properties simultaneously - security, decentralization, and human-meaningful names. However, challenges remain: reliance on users to store and maintain private keys securely. Decentralization may not be appropriate for every scenario - centralized systems with well-established reputations often work better in many cases.

EXAM TIP: Zooko's Triangle is guaranteed to appear in exams - memorize the 3 properties: Secure + Decentralized + Human-meaningful. Key statement: a system can NORMALLY achieve only TWO of these three. Namecoin resolves this using blockchain. Always mention Microsoft ION built on Bitcoin as the prime real-world example of decentralized identity.

Q8. What is Decentralized Finance (DeFi)? Discuss its motivation, advantages, disadvantages, use cases, and challenges.

ANSWER

1. Definition

DeFi: A paradigm of financial services built on blockchain where participants conduct business directly with each other WITHOUT the involvement of any intermediaries (banks, financial firms, etc.).

DeFi could be the 'killer app of the decentralized revolution'. Traditionally, finance requires a trusted third party (bank or financial firm) for every transaction. DeFi eliminates this requirement. At the time of writing, nearly $5 billion USD of value was locked in the DeFi system.

2. Problems with Traditional Finance (Why DeFi?)

- Access Barrier: KYC and rigorous onboarding create barriers for millions of unbanked people worldwide, especially in developing countries
- High Cost: Financial services (especially investment-related) can be costly - acts as a barrier to entry
- Transparency Issues: Concerns around transparency and trust due to the proprietary nature of the financial industry
- Siloed: Most financial solutions are proprietary and owned by their organization - causing interoperability issues between systems

3. Advantages of DeFi

- Financial Inclusion: No KYC barriers; accessible to anyone with an internet connection and a blockchain wallet
- Easy Access: Open to all - no geographic or regulatory gatekeeping
- Cheaper Services: No intermediary fees; lower transaction costs
- Transparency: All transactions verifiable on public blockchain
- No Single Authority: Participants conduct business directly - no central entity in control
- Interoperability: DeFi protocols can interact with each other (composability)

4. Challenges / Disadvantages of DeFi

- Underdeveloped Ecosystem: Still developing; requires more effort to improve usability and mainstream adoption
- Too Technical: Complex for average users - understanding financial transactions and jargon on DeFi platforms is daunting
- Lack of Regulation: No regulatory framework - can be used for illegal activities; trusting code instead of established financial institutions is a major concern
- Human Error: On a blockchain, human errors have serious, irreversible implications - financial loss cannot be recovered; no regulatory protection compared to traditional finance

NOTE: On human error: Blockchain is often perceived as tamper-proof and all-secure. This can give the wrong impression. Users may accept everything as accurate without understanding limitations. Validation checks must be implemented at UI level - e.g., verifying address format before sending funds. Funds sent to the wrong address CANNOT be recovered.

5. DeFi Use Cases

- Loans: Decentralized lending and borrowing without banks
- Decentralized Exchanges (DEXs): Trade crypto assets P2P without a centralized exchange
- Derivatives: Decentralized derivative contracts
- Payments: Cross-border payments without correspondent banks
- Insurance: Smart contract-based automated insurance payouts
- Assets: Tokenization of real-world assets

All use cases share one thing in common: no central authority in control; participants conduct business directly without intermediaries.

EXAM TIP: Structure your answer: (1) Define DeFi, (2) 4 problems of traditional finance - Access, Cost, Transparency, Siloed, (3) Advantages, (4) 4 Challenges - underdeveloped, too technical, lack of regulation, human error, (5) 6 use cases. The $5 billion locked value statistic strengthens your answer. Always explain that human errors on blockchain are IRREVERSIBLE - this is a critical DeFi challenge.

QUICK REFERENCE: Key Terms and One-Line Definitions

Term	One-Line Definition
Narayanan Framework	4 Qs: What is decentralized? What level? Which blockchain? What security mechanism?
Atomicity	Transaction executes fully or not at all - ensures deterministic integrity
DHT (Distributed Hash Table)	Decentralized storage mechanism used in P2P systems like BitTorrent
IPFS	Juan Benet's system replacing HTTP; uses Kademlia DHT (storage) + Merkle DAG (searching)
Filecoin	Incentive protocol for IPFS - pays nodes to store data permanently via Bitswap mechanism
BigChainDB	Fast, linearly scalable decentralized database; complements IPFS and Ethereum
Mesh Network	P2P communication network without central hub (ISP); example: Firechat
Whisper Protocol	Decentralized communication protocol in Ethereum's ecosystem
Zooko's Triangle	Naming system can have max 2 of 3: Secure + Decentralized + Human-meaningful
Namecoin	Blockchain that resolved Zooko's Triangle by achieving all 3 properties
Smart Contract	Self-executing code on blockchain with embedded business logic
Autonomous Agent (AA)	AI software entity acting on owner's behalf with minimal/no human intervention
DO	Decentralized Organization - NOT autonomous; relies on human input; has legal status
DAO	Fully autonomous; code is governing entity; non-profit; no legal status; Ethereum pioneered
DAC	Like DAO but CAN earn profit via shares and pay dividends to participants

DAS	Entire society functioning on blockchain via complex smart contracts and DAOs
Type 1 DApp	Runs on its own dedicated blockchain (e.g., standard Ethereum DApps with ETH token)
Type 2 DApp	Uses existing blockchain + custom protocols/tokens (e.g., DAI, Golem, OMNI on Bitcoin)
Type 3 DApp	Uses protocols of Type 2 DApps (e.g., SAFE Network, USDT/Tether)
Web 1	Static read-only web (1989 onwards) - users could only consume content
Web 2	Interactive centralized web - current era (Twitter, Facebook, Gmail)
Web 3	Decentralized user-owned web via blockchain (Steemit, Status, IPFS, DeFi)
Decentralized Identity	Returns identity credential control to the individual; no central authority
Microsoft ION	Decentralized identity network built on Bitcoin blockchain by Microsoft
DeFi	Decentralized Finance - financial services without intermediaries on blockchain
DEX	Decentralized Exchange - P2P crypto asset trading without a centralized exchange
EOS	Blockchain with ~3,996 TPS; raised $4B ICO; aims to be a decentralized OS
Lisk	JS-based DApp platform using DPOS with 101 elected validator nodes
MaidSafe	Non-blockchain SAFE network; encrypted chunk storage; rejects duplicates; uses Safecoin
Firechat	Mesh network app allowing P2P iPhone communication without Internet connection

MUST-KNOW POINTS FOR EXAM

1. Narayanan's 4-question framework: (1) What? (2) Level? (3) Which Blockchain? (4) Security Mechanism? - Apply with money transfer -> Bitcoin -> Atomicity example.

2. The decentralized ecosystem has 4 layers (bottom to top): Communication -> Storage -> Blockchain (Processing) -> Identity/Finance/Wealth.

3. IPFS uses TWO technologies: Kademlia DHT (storage) + Merkle DAG (searching). Filecoin is the incentive layer via Bitswap mechanism.

4. Zooko's Triangle: A system can normally only have 2 of 3 properties - Secure, Decentralized, Human-meaningful. Namecoin resolves all 3.

5. DO is NOT autonomous (human input required). DAO IS autonomous (AI logic). DAC = autonomous + can earn profit. Only DO has established legal status.

6. The DAO hack: $168M raised -> code bug exploited -> ETH stolen -> Ethereum hard fork required. Opens debate on smart contract security and testing.

7. DApp Types: Type 1 = own blockchain; Type 2 = existing blockchain + custom tokens; Type 3 = uses Type 2 protocol. USDT/Tether is the best Type 3 example.

8. DApp requirements (Johnston 2015): (1) Open source + autonomous, (2) Cryptographically secured, (3) Cryptographic token, (4) Token generated by consensus.

9. Web evolution: Web 1 = Read-only | Web 2 = Read+Write (Centralized) | Web 3 = Read+Write+OWN (Decentralized via blockchain).

10. TPS comparison: Bitcoin ~7 TPS | Ethereum ~15 TPS | EOS ~3,996 TPS. EOS raised $4 billion in its ICO (2018).

11. DeFi: 4 problems of traditional finance = Access Barrier + High Cost + Transparency Issues + Siloed systems.

12. Human errors on blockchain are IRREVERSIBLE - funds sent to wrong address cannot be recovered. This is a critical DeFi risk. Validation checks at UI level are essential.

13. MaidSafe is NOT a blockchain - it is a decentralized autonomous SAFE network. Key innovation: automatically rejects duplicate files to save computing resources.

14. Pre-blockchain P2P systems (BitTorrent, Gnutella) were partially decentralized but failed due to lack of incentivization mechanism - blockchain solved this problem.

Symmetric Cryptography

Cryptographic Services | Hash Functions | SHA-256 | SHA-3 | Symmetric Keys | MACs

Q1. What is Cryptography? Explain the generic cryptographic model and the security services it provides.

ANSWER

1. Definition of Cryptography

Cryptography: The science of making information secure in the presence of adversaries. It operates under the assumption that limitless resources are available to adversaries.

Cipher: An algorithm used to encrypt or decrypt data so that if intercepted by an adversary, the data is meaningless to them without decryption, which requires a secret key.

Cryptography is primarily used to provide a confidentiality service. On its own, it cannot be considered a complete solution - rather it serves as a crucial building block within a more extensive security system. Securing a blockchain ecosystem requires many different cryptographic primitives: hash functions, symmetric key cryptography, digital signatures, and public key cryptography.

2. The Generic Cryptographic Model

A generic cryptographic model is shown in the following diagram:

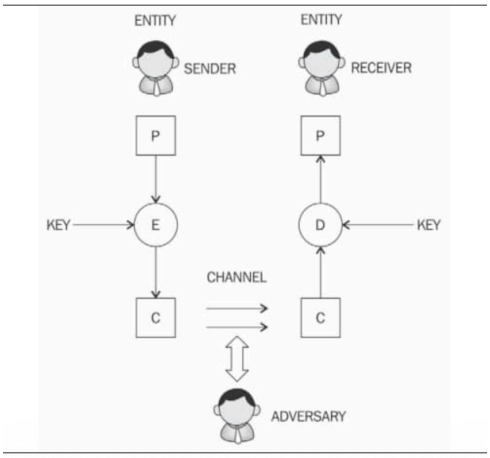

Figure 3.1: A model of the generic encryption and decryption model

The generic encryption/decryption model involves the following components:

Component	Symbol	Description
Plaintext	P	The original, readable message before encryption
Encryption	E	The process of converting plaintext to ciphertext using a key
Ciphertext	C	The encrypted, unreadable output transmitted over the channel
Decryption	D	The process of converting ciphertext back to plaintext using a key
Key	K	Secret data used for both encryption and decryption
Entity	-	A person or system that sends, receives, or operates on data
Sender	-	The entity that transmits the data
Receiver	-	The entity that receives the data
Adversary	-	An entity that tries to circumvent the security service

Channel	-	The medium of communication between entities

Flow: Sender takes Plaintext P, encrypts it using Key and Encryption function E to produce Ciphertext C. C is transmitted over the Channel. The Receiver decrypts C using the same Key and Decryption function D to recover Plaintext P. The Adversary intercepts C on the channel but cannot recover P without the key.

3. Security Services Provided by Cryptography

A. Confidentiality

Confidentiality: The assurance that information is only available to authorized entities.

- Achieved by encrypting data so that only authorized parties with the correct key can access it
- This is the primary service provided by cryptography

B. Integrity

Integrity: The assurance that information is modifiable only by authorized entities.

- Ensures that data has not been tampered with during transmission
- Commonly provided using hash functions and MACs

C. Authentication

Authentication: Provides assurance about the identity of an entity or the validity of a message.

Two types of authentication:

- Entity Authentication: Assurance that an entity is currently involved and active in a communication session
- Data Origin Authentication (Message Authentication): Assurance that the source of information is verified; guarantees data integrity because if source is corroborated, data must not have been altered

D. Non-Repudiation

Non-Repudiation: The assurance that an entity cannot deny a previous commitment or action by providing incontrovertible evidence.

- Produces cryptographic evidence in electronic transactions to confirm an action
- Essential in e-commerce disputes (e.g., placement of an order)
- Non-repudiation protocol requirements: fairness, effectiveness, and timeliness
- Multi-Party Non-Repudiation (MPNR): Protocols for scenarios with multiple participants (e.g., electronic trading with clearing agents, brokers, traders)

E. Accountability

Accountability: The assurance that actions affecting security can be traced back to the responsible party.

- Provided by logging and audit mechanisms
- Example: Electronic trading systems where a trade is logged with date, timestamp, and entity identity
- Log files can optionally be encrypted - stored as database entries or standalone ASCII text files

EXAM TIP: Always list ALL 5 services: Confidentiality, Integrity, Authentication (both types), Non-Repudiation, Accountability. The model components (P, E, C, D, Key, Channel, Adversary) are a common 5-mark question within a 10-mark question. Mention that cryptography alone is not a complete solution - it is a building block. MPNR is often asked as a definition.

Q2. Explain the types of Authentication in cryptography. Discuss Entity Authentication, Multi-Factor Authentication, and Data Origin Authentication.

ANSWER

1. Overview of Authentication

Authentication: Provides assurance about the identity of an entity or the validity of a message. Two main types: Entity Authentication and Data Origin Authentication.

2. Entity Authentication

Entity Authentication: The assurance that an entity is currently involved and active in a communication session.

Single-Factor Authentication

- Traditional approach: username and password
- Only one factor involved: something you KNOW (username + password)
- Not very secure - vulnerable to password leakage, phishing, brute force attacks

Multi-Factor Authentication (MFA)

Additional factors are now commonly used to provide better security. Use of additional techniques is known as multi-factor authentication (or two-factor authentication if only two methods are used).

The three authentication factors are:

Factor	Description	Examples
Something You KNOW	Knowledge-based factor	Password, PIN, secret question
Something You HAVE	Possession-based factor	Hardware token, smart card, OTP device, mobile phone
Something You ARE	Biometric-based factor	Fingerprint, retina scan, iris scan, hand geometry, voice recognition

Something You HAVE - Hardware Token

- User uses hardware token IN ADDITION to login credentials
- Both factors must be available to gain access - true two-factor authentication
- If hardware token is lost/stolen OR password forgotten - access is denied
- Hardware token alone is useless without the password (something you know)

Something You ARE - Biometrics

- Uses biometric features: fingerprint, retina, iris, or hand geometry
- Ensures the user was physically present during authentication - biometric features are unique to every individual
- Careful implementation required - some research shows biometric systems can be circumvented under specific conditions

3. Data Origin Authentication

Data Origin Authentication: Also known as message authentication. Assurance that the source of information is verified. Guarantees data integrity because if the source is corroborated, the data must not have been altered.

- Methods used: Message Authentication Codes (MACs) and digital signatures
- If a source is verified, the data cannot have been modified in transit
- Provides both authentication AND integrity simultaneously

EXAM TIP: The 3-factor model (Know, Have, Are) with examples is a very common question. Always mention that single-factor authentication (password only) is insecure due to password leakage. For biometrics, mention that careful implementation is required as biometric systems can be circumvented. Link data origin authentication to MACs and digital signatures.

Q3. Explain Cryptographic Primitives and their taxonomy. Discuss Keyless Primitives and the role of Random Numbers in cryptography.

ANSWER

1. Cryptographic Primitives

Cryptographic Primitives: The basic building blocks of a security protocol or system. Various cryptographic algorithms that are essential for building secure protocols and systems.

Security Protocol: A set of steps taken to achieve required security goals by utilizing appropriate security mechanisms. Types: authentication protocols, non-repudiation protocols, key management protocols.

2. Taxonomy of Cryptographic Primitives

Cryptographic primitives are divided into three main categories:

Category	Sub-Types	Description
Keyless Primitives	Random Numbers, Hash Functions	Do not require a key; provide foundational services
Symmetric Key Primitives	Secret Key Ciphers (Block Ciphers, Stream Ciphers), MACs	Use the same key for encryption and decryption
Asymmetric Key Primitives	Digital Signatures, Public Key Ciphers	Use a public-private key pair; covered in Chapter 4

This chapter covers Keyless Primitives and Symmetric Key Primitives. Asymmetric (Public Key) cryptography is covered separately.

3. Keyless Primitives - Random Numbers

Randomness: Provides an indispensable element for the security of cryptographic protocols. Used for the generation of keys and in encryption algorithms.

Private keys must be generated randomly so that they are unpredictable and not easy to guess. The measure of randomness is called ENTROPY.

Randomness ensures that operations of a cryptographic algorithm do not become predictable enough to allow cryptanalysts to predict outputs. Two categories of randomness sources exist:

A. Random Number Generators (RNGs)

RNG: Software or hardware systems that use randomness available in the real (analog) world.

- Sources: temperature variations, thermal noise from electronic components, acoustic noise
- Also: randomness from running computer processes (keystrokes, disk movements)
- Disadvantages: difficult to acquire data, not always available, limited availability, insufficient entropy
- This is called REAL randomness

B. Pseudorandom Number Generators (PRNGs)

PRNG: Deterministic functions that use a random initial value called a SEED to produce a random-looking set of elements.

- Commonly used to generate keys for encryption algorithms
- Example: Blum-Blum-Shub (BBS) is a common PRNG
- PRNGs are a BETTER alternative to RNGs due to their reliability and deterministic nature
- More practical: reliable, repeatable (given same seed), and computationally efficient

Feature	RNG	PRNG
Source	Real-world physical phenomena	Mathematical algorithm with a seed
Deterministic?	No - truly random	Yes - deterministic given same seed
Reliability	Lower - not always available	Higher - always produces output
Example	Thermal noise, keystrokes	Blum-Blum-Shub (BBS)
Practical Use	Limited due to acquisition difficulty	Preferred for key generation

EXAM TIP: Always mention ENTROPY as the measure of randomness. The BBS (Blum-Blum-Shub) PRNG should be mentioned by name. State clearly that PRNGs are BETTER than RNGs due to reliability and deterministic nature. The seed concept is important - same seed produces same sequence (deterministic). Examiners often ask to compare RNG vs PRNG.

Q4. What are Hash Functions? Explain their properties, design principles, families, and role in blockchain.

ANSWER

1. Definition

Hash Function: A keyless cryptographic function used to create fixed-length digests of arbitrarily long input strings. Provides a data integrity service.

Hash functions are usually built using iterated and dedicated hash function construction techniques. They can be used as one-way functions and to construct other cryptographic primitives such as MACs and digital signatures. Some applications use hash functions to generate PRNGs.

2. Practical Properties (2 Properties)

Property 1: Compression of Arbitrary Messages into Fixed-Length Digests

- Must accept input text of ANY length
- Must output a FIXED-LENGTH compressed message
- Output sizes: usually between 128-bits and 512-bits
- Example: SHA-256 always produces a 256-bit output regardless of input size

Property 2: Easy to Compute

- Hash functions must be efficient and fast one-way functions
- Must be very quick to compute regardless of message size
- Efficiency may decrease for very large messages, but still fast enough for practical use

3. Security Properties (3 Properties)

Property 1: Pre-Image Resistance (One-Way Property)

Given the equation:

```
h(x) = y
```

Given y (the hash output), it must be computationally infeasible to find x (the original input). x is called the pre-image of y. This is also called the one-way property.

- Cannot reverse-compute from hash output to original input
- Example: Given a Bitcoin address (hash), cannot derive the private key

Property 2: Second Pre-Image Resistance (Weak Collision Resistance)

Given x and h(x), it must be computationally infeasible to find any other message m where:

```
m != x  AND  h(m) = h(x)
```

- Given a known input and its hash, cannot find another different input that produces the same hash
- Also known as weak collision resistance

Property 3: Collision Resistance (Strong Collision Resistance)

Two different input messages must NOT hash to the same output:

```
h(x) != h(z)  for all x != z
```

- No two different inputs should produce the same hash output
- Also known as strong collision resistance
- Hash functions will ALWAYS have some collisions by mathematical necessity (pigeonhole principle), but they must be computationally impractical to find

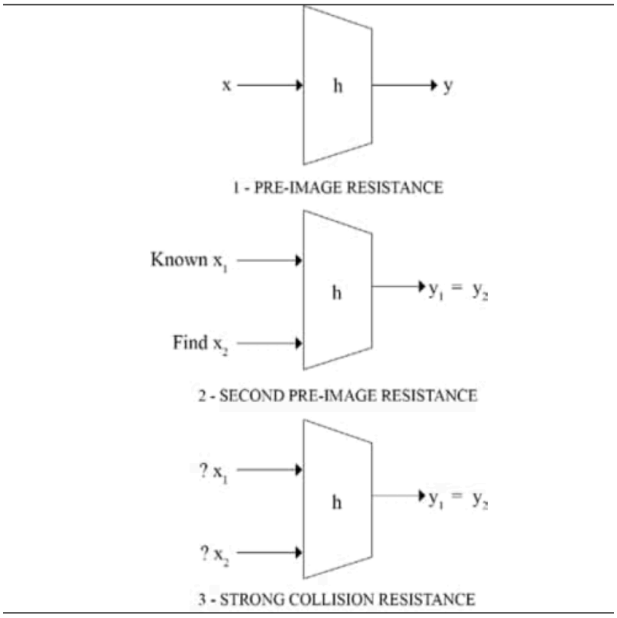

Figure 3.3: Three security properties of hash functions

Property	Also Known As	Formal Statement	Practical Meaning
Pre-Image Resistance	One-way property	Given h(x)=y, cannot find x	Cannot reverse a hash to find original input
Second Pre-Image Resistance	Weak collision resistance	Given x and h(x), cannot find m != x with h(m)=h(x)	Cannot find a different input with same hash as a known input
Collision Resistance	Strong collision resistance	Cannot find any x and z where h(x)=h(z)	Cannot find ANY two inputs with the same hash output

4. The Avalanche Effect

Avalanche Effect: A small change - even a single character change in the input text - will result in an entirely different hash output.

This property is desirable in all cryptographic hash functions and makes it practically impossible to predict input from output or to find related inputs with similar outputs.

OpenSSL Example demonstrating the avalanche effect:

```
$ echo -n 'Hello' | openssl dgst -sha256 (stdin)=
185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969  $ echo -n
'hello' | openssl dgst -sha256 (stdin)=
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
```

Note: Simply changing 'H' to 'h' produces a completely different 256-bit hash - demonstrating the avalanche effect.

5. Design - Iterated Hash Functions and Merkle-Damgard Construction

Hash functions are usually designed using the iterated hash functions approach:

- Input message is compressed in multiple rounds on a block-by-block basis

  Merkle-Damgard Construction: A popular iterated hash function construction. Divides input data into equal block sizes and feeds them through compression functions iteratively. The collision resistance of the compression functions ensures that the hash output is also collision-resistant.

- Compression functions can be built using block ciphers
- Other constructions: Miyaguchi-Preneel and Davies-Meyer

6. Hash Function Families

Family	Bit Size	Status	Notes
MD4	128-bit	INSECURE - deprecated	Message Digest family; early 1990s; not recommended
MD5	128-bit	INSECURE - deprecated	Commonly used for file integrity checks; now broken
SHA-0	160-bit	INSECURE	Introduced by NIST in 1993; replaced by SHA-1
SHA-1	160-bit	INSECURE - being deprecated	Introduced 1995; used in SSL/TLS; discouraged in new implementations
SHA-2 (SHA-224/256/384/512)	224-512 bit	SECURE - widely used	Four variants; SHA-256 used in Bitcoin PoW
SHA-3 (Keccak)	224-512 bit	SECURE - latest standard	Sponge construction; used in Ethereum; NIST standardized
RIPEMD-160	160-bit	SECURE	Based on MD4 design; used to produce Bitcoin addresses
Whirlpool	512-bit	SECURE	Based on Rijndael (W); uses Miyaguchi-Preneel compression

7. Hash Functions in Blockchain

- Proof-of-Work (PoW) in Bitcoin: Uses SHA-256 TWICE to verify computational effort of miners
- Bitcoin Addresses: RIPEMD-160 is used to produce Bitcoin addresses

- Ethereum: Uses Keccak (the original algorithm presented to NIST, rather than the modified NIST SHA-3 standard)
- Applications: Hash tables, distributed hash tables (DHT), bloom filters, virus fingerprinting, P2P file sharing, password storage, digital signatures

EXAM TIP: The 3 security properties are the most important topic in this question. Always use the formal notation: h(x)=y for pre-image, h(m)=h(x) for second pre-image, h(x)=h(z) for collision. The avalanche effect must be defined clearly. In blockchain context: Bitcoin uses SHA-256 (PoW) twice; Bitcoin addresses use RIPEMD-160; Ethereum uses Keccak (NOT the standard NIST SHA-3). Mention MD5 and SHA-1 are now insecure.

Q5. Explain the design of SHA-256. Discuss its structure, working steps, and its application in Bitcoin.

ANSWER

1. Overview of SHA-256

SHA-256 belongs to the SHA-2 family and is the most widely used hash function in blockchain. It is used in Bitcoin's Proof-of-Work (PoW) algorithm.

Parameter	Value
Maximum Input Message Size	2^64 - 1 bits
Block Size	512 bits
Word Size	32 bits
Output Digest Size	256 bits
Number of Rounds	64
Initial Hash Value	Eight 32-bit words (256 bits total)

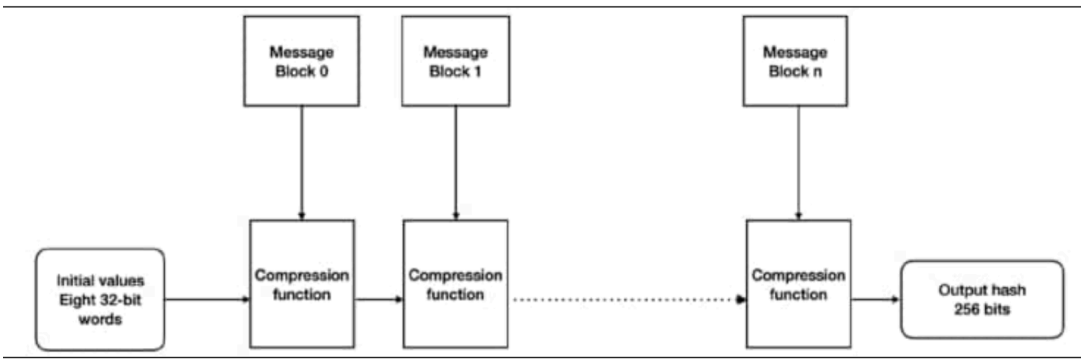

Figure 3.4: SHA-256 high level overview

2. Construction Type

Merkle-Damgard Construction: SHA-256 uses this construction. Input message is divided into equal 512-bit blocks (chunks) and fed through the compression function iteratively. Initial values (8 x 32-bit words = 256 bits) are fed into the compression function with the first message block. Each subsequent block is processed until all blocks are done and the final 256-bit hash is produced.

Initial Hash Values: Obtained from the first 32 bits of the fractional parts of the SQUARE ROOTS of the first EIGHT PRIME NUMBERS. These are fixed and chosen to ensure no backdoor exists in the algorithm.

3. Working Steps of SHA-256 (9 Steps)

Pre-processing Phase

- Step 1 - Padding: Message is padded to adjust the length of a block to 512 bits if it is smaller than the required block size
- Step 2 - Parsing: Message and its padding are divided into equal 512-bit blocks
- Step 3 - Initial Hash Value Setup: Eight 32-bit words obtained from first 32 bits of fractional parts of square roots of first eight prime numbers

Hash Computation Phase

- Step 4 - Block Processing: Each 512-bit message block is processed sequentially; 64 rounds required per block; each round uses slightly different constants to ensure no two rounds are identical
- Step 5 - Message Schedule Preparation
- Step 6 - Initialize Eight Working Variables (a, b, c, d, e, f, g, h)
- Step 7 - Run Compression Function 64 times
- Step 8 - Calculate Intermediate Hash Value
- Step 9 - Output: After repeating Steps 5-8 for ALL blocks, output hash is produced by concatenating intermediate hash values

4. SHA-256 Compression Function

In one round of the SHA-256 compression function:

- Registers: a, b, c, d, e, f, g, h are the 8 working variables
- Maj(a,b,c): Majority function - applied bitwise
- Ch(e,f,g): Choice function - applied bitwise
- Sigma-0 (S0) and Sigma-1 (S1): Perform bitwise rotation (diffusion)
- Round constants Wj and Kj: Added in the main loop (compression function) which runs 64 times
- The compression function processes a 512-bit message block with a 256-bit intermediate hash value

The compression function of SHA-256 is shown in the following diagram:

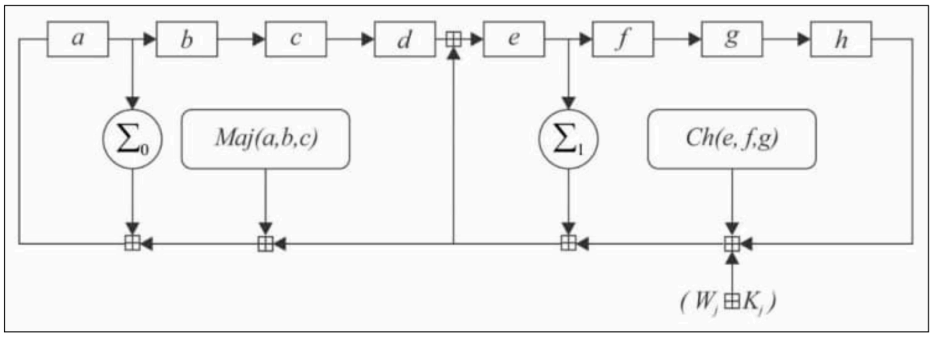

Figure 3.5: One round of an SHA-256 compression function

5. Application in Bitcoin

- Bitcoin's PoW (Proof-of-Work) algorithm uses SHA-256 TWICE (double-SHA-256)
- Miners repeatedly hash the block header (with a varying nonce) until the hash output is below the difficulty target
- This double application provides extra security against length-extension attacks

NOTE: Ethereum uses Keccak - the original algorithm presented to NIST - rather than the NIST standardized SHA-3. The NIST modifications included an increase in the number of rounds and simpler message padding.

EXAM TIP: Key numbers to memorize: Input limit = 2^64 - 1 bits; Block size = 512 bits; Word size = 32 bits; Output = 256 bits; Rounds = 64; Initial values = 8 x 32-bit words. Initial values are from square roots of the first 8 prime numbers - this is a commonly asked detail. Always mention that Bitcoin uses SHA-256 TWICE in PoW.

Q6. Explain the design of SHA-3 (Keccak). How does it differ from SHA-256? Describe the Sponge and Squeeze construction.

ANSWER

1. Overview of SHA-3 (Keccak)

SHA-3 is the latest family of SHA functions. It is a NIST-standardized version of Keccak. Ethereum uses Keccak (the original algorithm presented to NIST) rather than the NIST standard SHA-3, as NIST modified the original Keccak (increased number of rounds, simpler message padding).

SHA-3 variants: SHA3-224, SHA3-256, SHA3-384, SHA3-512, SHAKE128, SHAKE256.

SHAKE128 and SHAKE256: Extendable-Output Functions (XOFs) that allow the output to be extended to any desired length.

2. Key Differences from SHA-1 and SHA-2

Feature	SHA-1 / SHA-2	SHA-3 (Keccak)
Permutation Type	Keyed permutations	Unkeyed permutations
Construction	Merkle-Damgard transformation	Sponge and Squeeze construction (newer approach)
Model	Compression-function based	Random permutation model
Structure	Linear iterative compression	Absorb phase + Squeeze phase
Blockchain Use	SHA-256 used in Bitcoin PoW	Keccak used in Ethereum

3. The Sponge and Squeeze Construction

Sponge Construction: A new approach used in SHA-3/Keccak. Analogous to a physical sponge - data is first absorbed into the sponge, then the output is squeezed out.

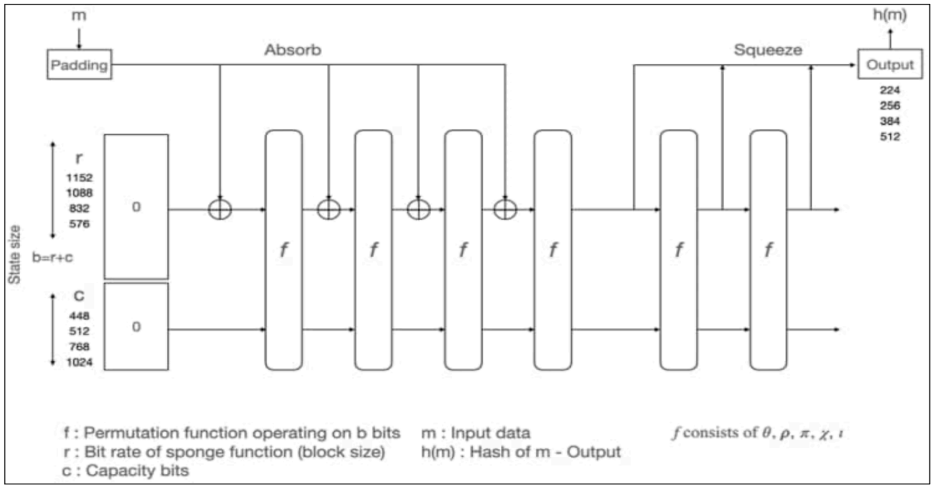

Figure 3.6: The SHA-3 absorbing and squeezing function in SHA3

The two phases:

Phase 1 - Absorb

- Data (m input data) is first padded
- Padded data is fed into the sponge block by block via XOR (exclusive OR)
- Changed into a subset of permutation state using XOR
- The rate r is the INPUT BLOCK SIZE of the sponge function

Phase 2 - Squeeze

- The output h(m) is squeezed out of the sponge function
- Represents the transformed state
- Capacity c DETERMINES THE SECURITY LEVEL
- State size b = r + c (bit rate + capacity)

4. State Size and Parameters

State size b is calculated by adding bit rate r and capacity c: b = r + c

Valid state sizes (r + c): 25, 50, 100, 200, 400, 800, or 1600 bits

The state is a 3-dimensional bit matrix. Initial state is set to 0.

r (Block Size)	c (Capacity)	Output Hash Size
1152	448	224 bits
1088	512	256 bits
832	768	384 bits
576	1024	512 bits

5. The Permutation Function f - Five Transformation Operations

The function f is a permutation function operating on b bits. It contains 5 transformations that combine to form one round. SHA-3 uses 24 rounds:

Operation	Symbol	Function
Theta	θ (Theta)	XOR bits in the state - used for mixing
Rho	ρ (Rho)	Diffusion function - performs rotation of bits
Pi	π (Pi)	Diffusion function
Chi	χ (Chi)	XOR each bit - bitwise combine
Iota	ι (Iota)	Combination with round constants

These five operations combined form ONE round. The number of rounds in SHA-3 standard is 24 to achieve the desired level of security. The goal of combining these five operations is to achieve the avalanche effect.

6. Key Insight on f Functions in SHA-3

- f = permutation function consisting of θ, ρ, π, χ, ι
- f operates on b bits (the full state)
- The composition: f consists of Theta, Rho, Pi, Chi, Iota applied in sequence per round
- 24 rounds per hash computation in standard SHA-3

EXAM TIP: The key difference between SHA-256 and SHA-3 is the construction: Merkle-Damgard (SHA-256) vs Sponge-and-Squeeze (SHA-3). Memorize the parameter table: r=1088, c=512 gives 256-bit output. Know the 5 operations: θ (mixing), ρ (rotation/diffusion), π (diffusion), χ (XOR/bitwise), ι (round constants). Number of rounds = 24. Ethereum uses Keccak (original), NOT the NIST standard SHA-3.

Q7. What is Symmetric Cryptography? Explain the types of cryptographic keys, key generation methods, and important concepts like Nonce, IV, and Salt.

ANSWER

1. Definition of Symmetric Cryptography

Symmetric Cryptography: A type of cryptography where the SAME key is used for both encrypting and decrypting the data. Also known as shared key cryptography or secret key cryptography.

The key must be established or agreed upon BEFORE data exchange occurs between the communicating parties. In symmetric key systems, only one key is required, which is shared between the sender and the receiver.

2. Types of Cryptographic Keys

Key Type	Description	Usage
Symmetric / Secret Key	Single shared key used for both encryption and decryption	Symmetric cryptography (AES, DES, 3DES)
Public Key	One half of an asymmetric key pair; can be shared openly	Encrypting plaintext in asymmetric cryptography
Private Key	Other half of asymmetric key pair; must be kept secret by receiver	Decryption in asymmetric cryptography; digital signatures
Ephemeral Key	Temporary key intended for short-term use (e.g., a single session)	Session keys in TLS/SSL protocols
Static Key	Long-term key intended for extended use over time	Long-term encryption relationships
Master Key	Used for protection, encryption, decryption, and generation of other keys	Key management systems

3. Methods of Key Generation

Method 1: Random Generation

- A random number generator is used to generate a random set of bytes
- The random bytes are used directly as a key
- Security depends entirely on quality of randomness (entropy)

Method 2: Key Derivation

- A single key or multiple keys are derived from a password

Key Derivation Function (KDF): Takes a password as input and converts it into a cryptographic key

- Common KDFs: PBKDF1, PBKDF2, Argon2, Scrypt
- Application: Ethereum wallets (keystore files) use Scrypt KDF to generate an AES symmetric key that encrypts the wallet

Method 3: Key Agreement Protocol

- Two or more participants run a protocol that produces a shared key
- All participants contribute equally to generate the shared secret key

Diffie-Hellman Key Exchange: The most commonly used key agreement protocol - allows two parties to establish a shared secret over an insecure channel without prior communication

4. Important Cryptographic Number Concepts

A. Nonce

Nonce: A number that can be used ONLY ONCE in a cryptographic protocol. Must not be reused.

- Generated from a large pool of random numbers OR can be sequential
- Most common use: prevent REPLAY ATTACKS in cryptographic protocols
- In blockchain: Bitcoin block header contains a nonce that miners vary to find a valid hash

B. Initialization Vector (IV)

IV (Initialization Vector): A random number (essentially a nonce) that must be chosen in an UNPREDICTABLE manner - cannot be sequential.

- Used extensively in encryption algorithms to provide increased security
- Difference from nonce: IV must be non-sequential/unpredictable; nonces can be sequential
- Used in block cipher modes of operation (CBC, CTR, GCM, etc.)

C. Salt

Salt: A cryptographically strong random value typically used in hash functions to provide defense against dictionary attacks and rainbow table attacks.

- Makes each password unique even if users have the same password
- Without salt: attacker can precompute hash of millions of dictionary words and match
- With salt: attacker must run a SEPARATE dictionary attack for each unique random salt - computationally infeasible
- Used in password storage: hash(password + salt) stored, not just hash(password)

Concept	Definition	Key Property	Primary Use
Nonce	Used only once in a protocol	Can be sequential	Prevent replay attacks
IV (Init Vector)	Random starting value for encryption	Must be non-sequential/unpredictable	Block cipher modes of operation
Salt	Random value added to password before hashing	Unique per password	Defense against dictionary/rainbow attacks

EXAM TIP: Always distinguish between the three random values: Nonce (sequential OK, prevents replay), IV (must be unpredictable/non-sequential, used in encryption), Salt (used with passwords to prevent dictionary attacks). The Ethereum wallet connection (Scrypt KDF -> AES key -> encrypt wallet) is a highly specific and frequently tested detail. Diffie-Hellman is the most common key agreement protocol.

Q8. What are Message Authentication Codes (MACs) and HMACs? How do they differ from hash functions, and what security services do they provide?

ANSWER

1. Background - From Hash Functions to MACs

Hash functions do not use a key. However, if hash functions are used WITH a key, they can be used to create another cryptographic construct called Message Authentication Codes (MACs).

MACs and HMACs are introduced after symmetric key cryptography because these constructions make use of keys for their operation.

2. Message Authentication Codes (MACs)

MAC: A keyed hash function that uses a symmetric (shared) key to provide both data integrity AND data origin authentication simultaneously.

- Unlike plain hash functions, MACs require a secret key known to both sender and receiver
- Provides AUTHENTICATION - verifies both the source and integrity of a message
- If the MAC value matches at the receiver end, the message is authenticated and unmodified
- MACs do NOT provide non-repudiation (because both parties share the same key)

3. How MACs Work

```
MAC = f(Key, Message)
```

Process:

- Sender computes MAC = f(Key, Message) using the shared secret key
- Sender sends Message + MAC to receiver
- Receiver computes MAC' = f(Key, Message) independently
- If MAC = MAC', message is authentic and unmodified
- If MAC != MAC', message has been tampered with or is from an unauthorized source

4. HMAC - Hash-based Message Authentication Code

HMAC: A specific type of MAC that uses a cryptographic hash function (such as SHA-256) in combination with a secret key. Standardized and widely used.

```
HMAC(K, m) = H((K XOR opad) || H((K XOR ipad) || m))
```

- H is the hash function (e.g., SHA-256, SHA-3)
- K is the secret key
- opad is the outer padding constant (0x5c repeated)
- ipad is the inner padding constant (0x36 repeated)
- HMAC applies the hash function TWICE for added security
- Common uses: TLS/SSL, API authentication, password verification

5. Comparison: Hash Function vs MAC vs HMAC

Feature	Hash Function	MAC	HMAC
Key Required?	No (keyless)	Yes (symmetric key)	Yes (symmetric key)
Integrity?	Yes	Yes	Yes
Authentication?	No	Yes	Yes
Non-Repudiation?	No	No	No
Example	SHA-256, RIPEMD-160	CMAC, GMAC	HMAC-SHA256, HMAC-SHA3
Standard	NIST FIPS 180	NIST SP 800-38	NIST FIPS 198

EXAM TIP: MACs provide BOTH integrity AND authentication (two services together). Hash functions provide only integrity. MACs do NOT provide non-repudiation because the same key is shared by both parties - either party could have created the MAC. HMAC applies the hash function TWICE using ipad and opad constants. Link MACs to data origin authentication from the cryptographic services topic.

QUICK REFERENCE: Key Terms and One-Line Definitions

Term	One-Line Definition
Cryptography	Science of making information secure in the presence of adversaries with limitless resources
Cipher	Algorithm used to encrypt/decrypt data; meaningless without decryption key
Plaintext (P)	Original readable message before encryption
Ciphertext (C)	Encrypted unreadable output transmitted over the channel
Confidentiality	Assurance that information is available ONLY to authorized entities
Integrity	Assurance that information is modifiable ONLY by authorized entities
Authentication	Assurance about identity of an entity or validity of a message
Non-Repudiation	Entity cannot deny a previous action - incontrovertible cryptographic evidence provided
Accountability	Actions affecting security can be traced back to the responsible party
MPNR	Multi-Party Non-Repudiation - protocols for transactions with multiple participants
Cryptographic Primitive	Basic building block of a security protocol or system
Entropy	The measure of randomness in a cryptographic context
RNG	Uses real-world physical phenomena for true randomness; less practical
PRNG	Deterministic algorithm using a seed; better alternative to RNG (reliability)
BBS	Blum-Blum-Shub - a common PRNG example
Hash Function	Keyless function producing fixed-length digest from arbitrary-length input
Pre-Image Resistance	Given h(x)=y, cannot reverse-compute x from y (one-way property)
Second Pre-Image Resistance	Given x and h(x), cannot find m!=x with h(m)=h(x) - weak collision resistance
Collision Resistance	Cannot find any x,z where h(x)=h(z) - strong collision resistance
Avalanche Effect	Small input change causes completely different hash output
Merkle-Damgard	Iterated hash construction; divides input into 512-bit blocks; used in SHA-256
SHA-256	512-bit blocks, 256-bit output, 64 rounds, 8 x 32-bit initial values; used in Bitcoin PoW
SHA-3 / Keccak	Sponge-and-Squeeze construction, unkeyed permutations, 24 rounds; used in Ethereum
Sponge Construction	Data absorbed block by block via XOR, then output squeezed out; basis of SHA-3

XOF	Extendable-Output Function - SHAKE128 and SHAKE256 variants of SHA-3
Symmetric Cryptography	Same key used for both encryption and decryption; also called secret/shared key cryptography
Ephemeral Key	Temporary key for short-term use (e.g., one session)
Master Key	Used for protection, encryption, decryption, and generation of other keys
KDF	Key Derivation Function - converts a password into a cryptographic key
Scrypt	KDF used in Ethereum wallets to generate AES symmetric key for wallet encryption
Diffie-Hellman	Most common key agreement protocol - establishes shared secret over insecure channel
Nonce	Number used ONLY ONCE in a protocol; can be sequential; prevents replay attacks
IV (Init Vector)	Random non-sequential starting value; must be unpredictable; used in encryption algorithms
Salt	Random value added to password before hashing; prevents dictionary/rainbow table attacks
MAC	Keyed hash function providing both integrity AND data origin authentication
HMAC	Hash-based MAC; applies hash function twice using ipad/opad constants with secret key

MUST-KNOW POINTS FOR EXAM

1. Cryptography provides 5 services: Confidentiality, Integrity, Authentication (Entity + Data Origin), Non-Repudiation, Accountability.

2. Three factors of Multi-Factor Authentication: Something you KNOW (password) + Something you HAVE (hardware token) + Something you ARE (biometrics).

3. Three types of cryptographic primitives: Keyless (Random numbers + Hash functions) | Symmetric key (Block ciphers + Stream ciphers + MACs) | Asymmetric key (Digital signatures + Public key ciphers).

4. Entropy = measure of randomness. PRNGs are BETTER than RNGs (more reliable, deterministic). BBS = Blum-Blum-Shub = common PRNG example.

5. Three security properties of hash functions: (1) Pre-image resistance = one-way; (2) Second pre-image resistance = weak collision; (3) Collision resistance = strong collision.

6. Avalanche effect: even a single character change in input produces a completely different hash output. Essential for security.

7. SHA-256 specs: Input limit = 2^64-1 bits | Block = 512 bits | Word = 32 bits | Output = 256 bits | Rounds = 64 | Initial values = 8 x 32-bit words from sqrt of first 8 primes.

8. SHA-3 (Keccak) uses Sponge-and-Squeeze construction (NOT Merkle-Damgard). 24 rounds. 5 operations: Theta(mixing), Rho(rotation), Pi(diffusion), Chi(XOR), Iota(round constants).

9. Blockchain hashing: Bitcoin PoW uses SHA-256 TWICE. Bitcoin addresses use RIPEMD-160. Ethereum uses Keccak (original, not NIST SHA-3).

10. Symmetric cryptography = same key for encryption AND decryption. Also called secret key / shared key cryptography. Key must be agreed BEFORE data exchange.

11. Three number concepts: Nonce (once only, can be sequential, prevents replay attacks) | IV (must be unpredictable/non-sequential, used in encryption) | Salt (random, prevents dictionary/rainbow attacks on passwords).

12. Ethereum wallet uses Scrypt KDF to generate an AES symmetric key that encrypts the wallet keystore file.

13. Diffie-Hellman = most common key agreement protocol. PBKDF1, PBKDF2, Argon2, Scrypt = Key Derivation Functions.

14. MACs vs Hash functions: MACs need a key; provide BOTH integrity AND authentication. Hash functions are keyless; provide only integrity. Neither provides non-repudiation.

Public Key Cryptography

Asymmetric Cryptography | RSA | ECC | Elliptic Curves | Point Operations | Discrete Logarithm

---

Q1. Explain the mathematical foundations of Public Key Cryptography: Modular Arithmetic, Sets, Fields, Groups, Rings, and Cyclic Groups.

ANSWER

1. Modular Arithmetic

Modular Arithmetic: Also known as clock arithmetic. Numbers wrap around when they reach a fixed number called the modulus. All operations are performed with respect to this fixed modulus.

Analogy: A 12-hour clock. Numbers go from 1 to 12. When 12 is reached, they start from 1 again.

Example: If the time is 9:00, then 4 hours later it will be 1:00 (not 13:00). In modular arithmetic: 9 + 4 = 13, but 13 mod 12 = 1.

Core concept: Modular arithmetic deals with REMAINDERS after division.

```
50 mod 11 = 6    (because 50 / 11 = 4 remainder 6)
```

2. Sets

Set: A collection of distinct objects.

```
Example: X = {1, 2, 3, 4, 5}
```

3. Fields

Field: A set in which all its elements form an additive and multiplicative group. Satisfies specific axioms for addition and multiplication. The distributive law also applies.

- Distributive law: The same sum or product will be produced even if any terms or factors are reordered

Finite Fields (Galois Fields)

Finite Field: A field with a FINITE set of elements. Also known as Galois Fields.

- Of particular importance in cryptography - produce accurate and error-free arithmetic results
- Prime finite fields are used in Elliptic Curve Cryptography (ECC) to construct discrete logarithm problems

Prime Fields

Prime Field: A finite field with a PRIME number of elements. Addition and multiplication are performed modulo p (a prime number). Each non-zero element has an inverse.

4. Groups

Group: A commutative set with an operation that combines two elements of the set. The operation is closed and associated with a defined identity element. Each element has an inverse.

Four group axioms must be satisfied:

Axiom	Meaning
Closure	If A and B are in the set, the result of the operation on A and B is also in the set
Associativity	Grouping of elements does not affect the result of the operation
Identity Element	There exists an element I such that A * I = A for all elements A
Inverse Element	For each element A, there exists an element A^-1 such that A * A^-1 = Identity

Abelian Groups

Abelian Group: Formed when the GROUP OPERATION on elements is also COMMUTATIVE. Satisfies all 4 group axioms PLUS commutativity.

```
A x B = B x A    (commutative law)
```

Key difference: A group requires 4 axioms; an Abelian group requires 5 axioms (closure, associativity, identity, inverse, + commutativity).

5. Rings

Ring: Formed when MORE THAN ONE operation can be defined over an abelian group. A ring must have closure, associative, and distributive properties.

6. Cyclic Groups

Cyclic Group: A type of group that can be generated by a single element called the GROUP GENERATOR.

- All elements of the group can be obtained by repeatedly applying the group operation to the generator
- Cyclic groups are essential in ECC for constructing the discrete logarithm problem

7. Order

Order (Cardinality): The NUMBER OF ELEMENTS in a field. Also known as the cardinality of the field.

Mathematical Structure	Key Property	Relevance to Cryptography
Modular Arithmetic	Numbers wrap around at modulus; deals with remainders	Foundation of RSA and discrete logarithm problems
Finite Field (Galois Field)	Finite set of elements; error-free arithmetic	Used in ECC for discrete logarithm problems
Prime Field	Prime number of elements; operations mod p	Base field for Elliptic Curve Cryptography
Group	4 axioms: closure, associativity, identity, inverse	Basic algebraic structure for cryptographic operations
Abelian Group	Group + commutativity (5 axioms)	Foundation for elliptic curve group operations
Ring	Multiple operations on abelian group	Advanced algebraic structure in cryptography
Cyclic Group	Generated by a single generator element	Essential for constructing discrete logarithm in ECC

Order	Number of elements in the field	Determines security level of the cryptographic system

EXAM TIP: These mathematical concepts are frequently asked as 'define and explain' questions. Memorize the 4 group axioms and the 5 Abelian group axioms (4 + commutativity). The analogy of the clock for modular arithmetic always helps. Finite fields = Galois fields - both names are used. Cyclic groups generate ALL elements from a single generator - this is the key to understanding discrete logarithm.

Q2. What is Asymmetric (Public Key) Cryptography? Explain how it works, its security mechanisms, and the three categories of asymmetric algorithms.

ANSWER

1. Definition

Asymmetric Cryptography: A type of cryptography where the key used to ENCRYPT the data is DIFFERENT from the key used to DECRYPT the data. Also called public key cryptography.

- Uses a PUBLIC KEY for encryption and a PRIVATE KEY for decryption
- The two keys are mathematically related but computationally infeasible to derive one from the other
- Algorithms: RSA, DSA (Digital Signature Algorithm), ElGamal encryption

2. How Asymmetric Cryptography Works

An overview of public-key cryptography is shown in the following diagram:

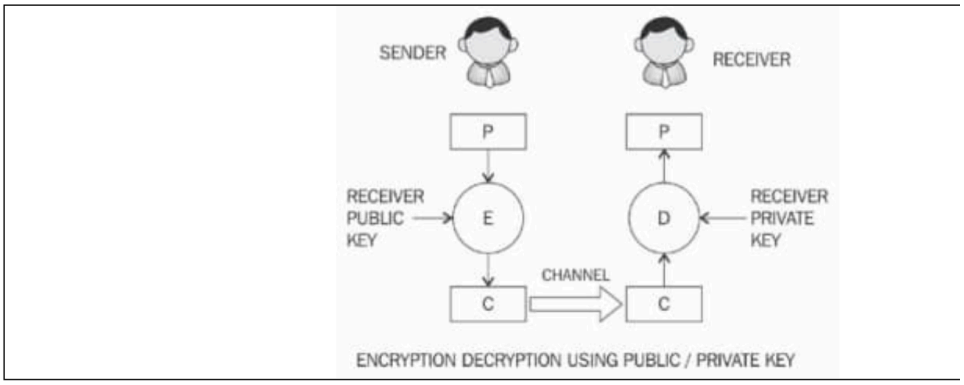

Figure 4.1: Encryption/decryption using public/private keys

Encryption/Decryption Model

- Sender encrypts plaintext P using RECEIVER'S PUBLIC KEY and encryption function E to produce ciphertext C
- C is transmitted over the network channel
- Receiver decrypts C using RECEIVER'S PRIVATE KEY with decryption function D to recover plaintext P
- Private key REMAINS on receiver's side - no key sharing required (unlike symmetric encryption)

```
C = E(Receiver_PublicKey, P)      (Encryption)
P = D(Receiver_PrivateKey, C)     (Decryption)
```

Digital Signature Model (Signing and Verification)

- Sender signs plaintext P using SENDER'S PRIVATE KEY and signing function S to produce signed data C
- Receiver verifies C using SENDER'S PUBLIC KEY and verification function V
- This model does NOT encrypt - it provides message authentication and integrity

```
C = S(Sender_PrivateKey, P)        (Signing)
P = V(Sender_PublicKey, C)         (Verification)
```

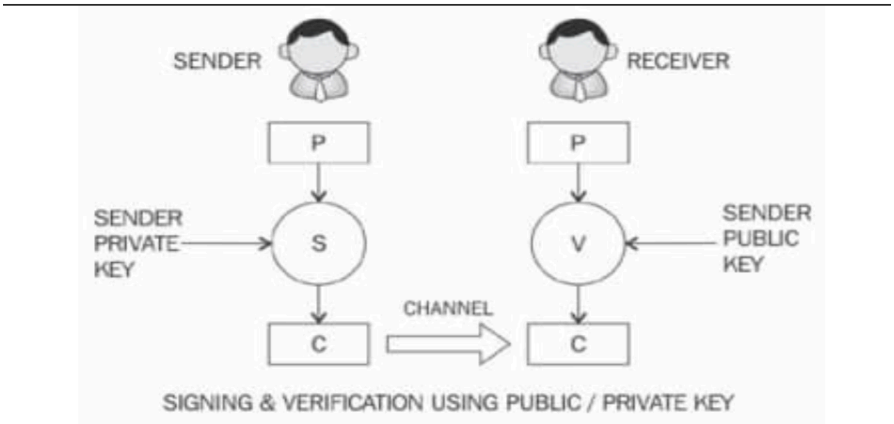

Figure 4.2: Model of a public-key cryptography signature scheme

3. Security Mechanisms of Public Key Cryptosystems

Security Mechanism	Description
Key Establishment	Design of protocols to set up keys over an insecure channel (e.g., Diffie-Hellman)
Digital Signatures	Provides non-repudiation - an action cannot be denied after it is taken
Identification	Entity identification using combination of digital signatures and challenge-response protocols
Encryption	Confidentiality using public key cryptosystems (RSA, ECC, ElGamal)
Decryption	Recovery of plaintext using private key

4. Key Limitation of Asymmetric Cryptography

Public key algorithms are SLOWER in terms of computation than symmetric key algorithms. Therefore:

- NOT used for encrypting large files or bulk data
- Typically used to exchange/establish keys for symmetric algorithms
- Once keys are established securely, faster symmetric algorithms encrypt the actual data

5. Three Categories of Asymmetric Algorithms

A. Integer Factorization

- Based on the fact that large integers are VERY HARD TO FACTOR
- Multiplying two large primes is easy; factoring the result back is computationally infeasible
- Prime example: RSA algorithm

B. Discrete Logarithm

- Based on a problem in modular arithmetic
- Easy to calculate the result of a modulo function (exponentiation)
- Computationally IMPRACTICAL to find the exponent from the result (one-way function)

```
3^2 mod 10 = 9    (easy to compute)
Given 9, find exponent 2 of generator 3    (extremely hard)
```

- Used in: Diffie-Hellman key exchange and Digital Signature Algorithms

C. Elliptic Curves

- Based on the discrete logarithm problem but in the context of ELLIPTIC CURVES
- An elliptic curve is an algebraic cubic curve over a field
- Non-singular curve (no cusps or self-intersections)
- Key schemes: ECDSA (Elliptic Curve Digital Signature Algorithm) and ECDH (Elliptic Curve Diffie-Hellman key exchange)

Category	Mathematical Problem	Key Algorithm	Used In
Integer Factorization	Hard to factor product of two large primes	RSA	Secure communications, TLS/SSL
Discrete Logarithm	Hard to find exponent given base and result in modular arithmetic	Diffie-Hellman, DSA, ElGamal	Key exchange, digital signatures
Elliptic Curves	Discrete log problem on elliptic curves over finite fields	ECDSA, ECDH	Bitcoin (ECDSA), Ethereum, blockchain generally

EXAM TIP: The two models (encryption/decryption vs. signing/verification) are commonly asked. Key insight: In encryption - use RECEIVER'S public key. In signing - use SENDER'S private key. For verification - use SENDER'S public key. Always mention that public key algorithms are SLOWER than symmetric algorithms and are used for KEY EXCHANGE, not bulk data encryption.

Q3. Explain Public and Private Keys in asymmetric cryptography. How are they generated and used? Compare symmetric and asymmetric cryptography.

ANSWER

1. Private Keys

Private Key: A randomly generated number that is kept SECRET and held privately by its user. Never shared with anyone.

- Must be protected - unauthorized access jeopardizes the entire public key cryptography scheme
- Used to: DECRYPT messages encrypted with the corresponding public key
- Used to: SIGN messages (producing digital signatures)
- Key length in RSA: Typically 1,024 bits or 2,048 bits
- Security note: 1,024-bit RSA keys are NO LONGER CONSIDERED SECURE; minimum 2,048-bit is recommended
- In ECC: Same security as 3,072-bit RSA achieved with only 256-bit ECC key

2. Public Keys

Public Key: Freely available and published by the private key owner. Anyone can use it to send encrypted messages to the key owner.

- Published openly - anyone can access and use it for encryption
- Used to: ENCRYPT messages to send to the private key holder
- Used to: VERIFY digital signatures created by the corresponding private key
- No need for secure key distribution (unlike symmetric cryptography)

Concerns with public keys:

- Authenticity: Is this really the correct public key for the intended recipient?
- Identification: How to verify the publisher of the public key is who they claim to be?
- Solution: Public Key Infrastructure (PKI) and digital certificates address these concerns

3. Symmetric vs. Asymmetric Cryptography - Comparison

Feature	Symmetric Cryptography	Asymmetric Cryptography
Keys Used	Single shared key for both encryption and decryption	Two keys: Public key (encrypt) and Private key (decrypt)
Key Distribution	Key must be shared securely BEFORE communication	Public key shared openly; private key stays secret
Speed	Fast - efficient for large data	Slow - computationally intensive
Use Case	Bulk data encryption (AES, DES)	Key exchange, digital signatures, authentication (RSA, ECC)
Key Size	Smaller (128-256 bits for AES)	Larger (2048 bits RSA; 256 bits ECC for equivalent security)
Security Model	Relies on secrecy of the single shared key	Relies on mathematical hardness (factoring, discrete log)
Non-Repudiation	NOT provided (both parties know the key)	PROVIDED via digital signatures
Examples	AES, DES, 3DES	RSA, ECC, DSA, ElGamal, Diffie-Hellman

EXAM TIP: The key difference in practice: Symmetric uses ONE key (shared secret); Asymmetric uses TWO keys (public + private). Asymmetric is slower, so in practice a HYBRID approach is used: asymmetric for key exchange, symmetric for actual data encryption (this is how TLS/SSL works). The 2,048-bit RSA vs 256-bit ECC equivalence is a commonly tested comparison.

Q4. Explain the RSA algorithm in detail. Discuss the key generation process, encryption, and decryption with relevant equations.

ANSWER

1. Overview of RSA

RSA: A public key cryptography algorithm invented in 1977 by Ron Rivest, Adi Shamir, and Leonard Adelman (hence RSA). Based on the INTEGER FACTORIZATION problem.

Core principle: Multiplying two large prime numbers is easy, but factoring the resulting product back to the original two prime numbers is computationally infeasible.

RSA was the FIRST implementation of public key cryptography.

2. RSA Key Generation Process (4 Steps)

Step 1: Modulus Generation

- Select p and q - two very large prime numbers (must be kept secret)
- Compute the modulus n by multiplying p and q:

```
n = p x q
```

- n forms part of both the public and private key

Step 2: Generate the Co-Prime (e)

- Choose a number e that satisfies:

```
1 < e < (p-1)(q-1)
```

- e must be CO-PRIME with (p-1)(q-1): no number other than 1 can divide both e and (p-1)(q-1)
- This means gcd(e, (p-1)(q-1)) = 1

Step 3: Generate the Public Key

- The PUBLIC KEY is the pair (n, e)
- This is shared publicly with anyone who wants to send encrypted messages
- p and q must be kept SECRET (knowing p and q allows calculation of private key d)

Step 4: Generate the Private Key (d)

- The PRIVATE KEY is d - the modular inverse of e

```
e x d = 1 mod (p-1)(q-1)
```

- d is calculated using the Extended Euclidean Algorithm (takes p, q, and e as input)
- Anyone who knows p and q can easily calculate d using the Extended Euclidean Algorithm
- Anyone who does NOT know p and q cannot generate d
- Therefore, p and q must be large enough to make n extremely difficult to factor

3. RSA Encryption

To encrypt plaintext P, the sender uses the receiver's public key (n, e):

```
C = P^e mod n
```

Plaintext P is raised to the power of e and then reduced modulo n to produce ciphertext C.

4. RSA Decryption

To decrypt ciphertext C, the receiver uses their private key d:

```
P = C^d mod n
```

The receiver raises C to the power of private key d and reduces modulo n to recover plaintext P.

5. RSA Summary Table

Component	Value/Formula	Who Knows It?
p, q	Two large prime numbers	Private - known only to key generator
n	n = p x q (modulus)	Public - part of public key
e	Co-prime of (p-1)(q-1); 1 < e < (p-1)(q-1)	Public - part of public key
Public Key	(n, e)	Freely shared with everyone
d (Private Key)	e x d = 1 mod (p-1)(q-1)	Private - kept secret by owner
Encryption	C = P^e mod n	Anyone with public key can encrypt
Decryption	P = C^d mod n	Only private key holder can decrypt

6. Security of RSA

- Security relies on the computational difficulty of FACTORING n back into p and q
- p and q must be large enough to make n computationally infeasible to factor

- Recommended minimum key size: 2,048 bits (1,024-bit is no longer secure)
- Extended Euclidean Algorithm computes d from p, q, e - this is why p and q must remain secret

EXAM TIP: Memorize the 4-step key generation: (1) n=p*q, (2) choose e co-prime to (p-1)(q-1), (3) public key=(n,e), (4) d = inverse of e mod(p-1)(q-1). Then memorize Encryption: C = P^e mod n and Decryption: P = C^d mod n. Always state that RSA was invented in 1977 by Rivest, Shamir, Adelman. The Extended Euclidean Algorithm computes d - this is the key detail about why p and q must stay secret.

Q5. What is Elliptic Curve Cryptography (ECC)? Explain its mathematical basis, advantages over RSA, and its applications in blockchain.

ANSWER

1. Definition of ECC

ECC: A public key cryptography system based on the DISCRETE LOGARITHM PROBLEM founded upon ELLIPTIC CURVES OVER FINITE FIELDS (Galois Fields).

The most prominent cryptosystems based on ECC are:

- ECDSA - Elliptic Curve Digital Signatures Algorithm (used in Bitcoin and Ethereum for signing transactions)
- ECDH - Elliptic Curve Diffie-Hellman (used for key exchange)

2. Key Advantage of ECC over RSA

The MAIN BENEFIT of ECC over RSA and other public key algorithms is that it requires a SMALLER KEY SIZE while providing the SAME LEVEL OF SECURITY.

Security Level	RSA Key Size Required	ECC Key Size Required	Advantage
Medium Security	2,048 bits	224 bits	9x smaller key
High Security	3,072 bits	256 bits	12x smaller key
Very High Security	7,680 bits	384 bits	20x smaller key

This smaller key size makes ECC ideal for:

- Embedded platforms and IoT devices (limited storage)
- Mobile devices and applications (battery and processing constraints)
- Blockchain transactions (efficiency and reduced storage requirements)

3. The Elliptic Curve Equation (Weierstrass Equation)

An elliptic curve is a type of polynomial equation known as the Weierstrass equation. It generates a curve over a finite field:

```
y^2 = x^3 + Ax + B
```

Where:

- A and B are integers belonging to a prime finite field Zp or Fp
- A special value called the POINT OF INFINITY exists to provide identity operations for points on the curve
- The curve is NON-SINGULAR: it has no cusps or self-intersections

Non-singular condition (must be satisfied):

```
4a^3 + 27b^2 != 0 (mod p)
```

This condition ensures the curve has no repeated roots and is non-singular.

4. Elliptic Curve Over Finite Fields

- For cryptographic purposes, the curve is defined over PRIME FINITE FIELDS (not real numbers)
- The prime p must be GREATER THAN 3
- Arithmetic operations are performed MODULO a prime number p
- Elliptic curve groups consist of POINTS ON THE CURVE over the finite field
- Different curves can be generated by varying the values of A and/or B

5. The Discrete Logarithm Problem in ECC

The discrete logarithm problem for ECC can be stated as:

```
Given y, find x such that g^x = y
```

- This is a ONE-WAY FUNCTION: easy to compute g^x but computationally infeasible to invert
- Given point T and base point P on the curve, it is easy to compute T = dP (scalar multiplication)
- But given T and P, it is computationally infeasible to find d (the private key)

6. Applications in Blockchain

- Bitcoin: Uses ECDSA (over curve secp256k1) for transaction signing and key generation
- Ethereum: Also uses ECDSA for transaction signing and address generation
- Private keys in blockchain are ECDSA private keys (256-bit scalars)
- Public keys are derived from private keys using elliptic curve point multiplication
- Blockchain addresses are derived from public keys via hashing (RIPEMD-160 for Bitcoin)

EXAM TIP: The key comparison to remember: ECC 256-bit provides security equivalent to RSA 3,072-bit. ECC is used in blockchain (Bitcoin and Ethereum use ECDSA). The equation is y^2 = x^3 + Ax + B. The non-singular condition 4a^3 + 27b^2 != 0 (mod p) must be mentioned. ECC is popular on embedded/mobile platforms due to smaller key size.

Q6. Explain the group operations on Elliptic Curves: Point Addition, Point Doubling, and Point Multiplication. Provide relevant equations.

ANSWER

1. Basic Group Operations on Elliptic Curves

Two basic group operations are defined on elliptic curves. These operations form the foundation of ECC:

- Point Addition: Two DIFFERENT points P and Q are added to get a third point R
- Point Doubling: A point P is added to ITSELF to get 2P

From these two operations, a third operation is derived:

- Point Multiplication (Scalar Point Multiplication): A point P is added to itself d times

2. Point Addition (P + Q = R)

Point Addition: A process where two different points P and Q on the elliptic curve are added to produce a third point R on the curve.

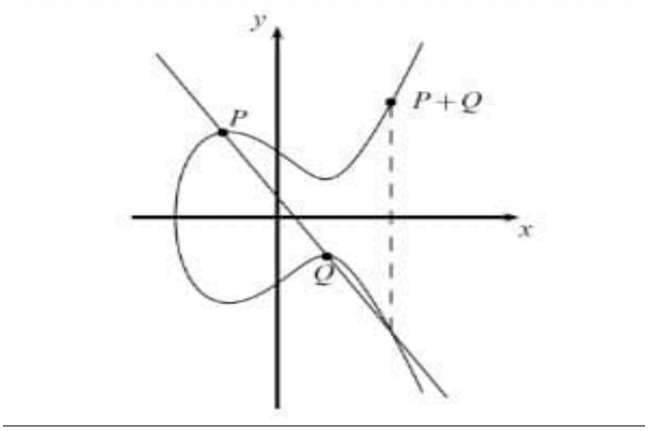

Figure 4.3: Point addition over R

Geometric interpretation:

- Draw a diagonal line (secant line) through points P and Q on the curve
- The line intersects the curve at a third point
- This third point is then MIRRORED (reflected across the x-axis) to yield R = P + Q

The result: (x1, y1) + (x2, y2) = (x3, y3)

Point addition equations (mod p arithmetic):

```
x3 = s^2 - x1 - x2  mod p
y3 = s(x1 - x3) - y1  mod p
```

Where S (the slope of the line through P and Q) is:

```
S = (y2 - y1) / (x2 - x1)  mod p
```

Worked Example over finite field F23, with y^2 = x^3 + 7x + 11:

- P = (12, 12), Q = (7, 9)
- After applying the point addition formulas: R = P + Q = (20, 20)
- There are 27 solutions to the equation over F23

3. Point Doubling (2P = P + P)

Point Doubling: A process where a point P is added to ITSELF. Occurs when P and Q are at the same point on the curve.

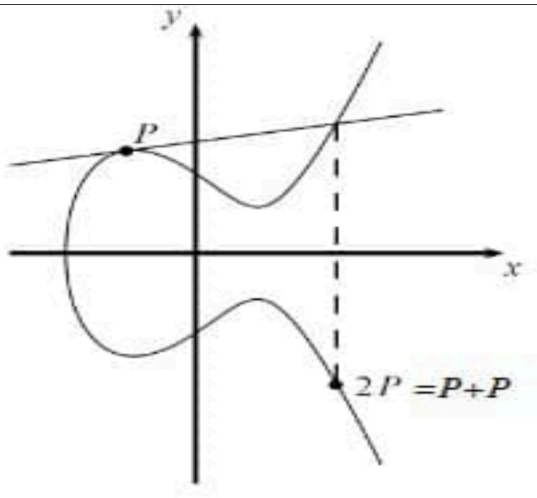

Figure 4.6: Graph representing point doubling

Geometric interpretation:

- Draw a TANGENT LINE at point P on the curve

- The tangent line intersects the curve at a second point
- This second point is MIRRORED to yield the result 2P = P + P

Point doubling equations (mod p arithmetic):

```
x3 = s^2 - x1 - x2  mod p
y3 = s(x1 - x3) - y1  mod p
```

Where S (the slope of the TANGENT LINE at P) is:

```
S = (3x1^2 + a) / (2y1)  mod p
```

Note: S uses the tangent slope formula (not secant), as P and Q are the same point.

Worked Example over finite field F23, with y^2 = x^3 + 7x + 11:

- P = (12, 11)
- After applying point doubling: 2P = R = (3, 17)

4. Point Multiplication (Scalar Point Multiplication)

Point Multiplication: Also called scalar point multiplication. Multiplying a point P by an integer d by repeatedly adding P to itself d times.

```
dP = P + P + P + ... + P  (P added d times)
```

- The result dP is always another point on the curve
- The operation of simple repeated addition becomes INEFFICIENT for large d
- Solution: The Double-and-Add Algorithm

The Double-and-Add Algorithm

An efficient method to compute dP by combining point addition and point doubling operations:

- Step 1: Convert d to binary
- Step 2: Starting from the most significant bit (MSB), skip the first bit
- Step 3: For each remaining bit: if bit = 1, perform DOUBLE and ADD; if bit = 0, perform DOUBLE only

Example: Computing 9P where 9 = 1001 in binary

- Binary: 1 0 0 1 (skip leftmost 1)
- Bit 0 (second from left): Double -> 2P
- Bit 0 (third): Double -> 4P
- Bit 1 (last): Double and Add -> 8P + P = 9P
- Result: Only 3 double operations + 1 addition = 4 operations instead of 9 additions
- This produces EXPONENTIAL PERFORMANCE GAIN for large values of d

5. Key Insight: The Security Foundation

Point multiplication forms the basis of ECC security:

- Given d (private key) and P (base/generator point): Computing T = dP is EASY (use double-and-add)
- Given T and P: Finding d is COMPUTATIONALLY INFEASIBLE (this is the ECDLP - Elliptic Curve Discrete Logarithm Problem)

Operation	Description	Line Type	Formula for S
Point Addition (P+Q)	Two different points added; line through both points intersects curve at third point; mirror to get R	Secant line through P and Q	S = (y2-y1)/(x2-x1) mod p
Point Doubling (2P)	Same point added to itself; tangent at P intersects curve at second point; mirror to get 2P	Tangent line at P	S = (3x1^2 + a)/(2y1) mod p

Point Multiplication (dP)	P added to itself d times; computed efficiently using double-and-add algorithm	N/A - combines both	Double: S from doubling; Add: S from addition

EXAM TIP: The most tested items: (1) Point addition uses a SECANT line; point doubling uses a TANGENT line. (2) The slope formulas are different for each. (3) The double-and-add example with 9P (1001 in binary = 3 doubles + 1 add instead of 9 additions) is a frequently asked calculation. (4) Security is based on the ECDLP: given T=dP, finding d is computationally infeasible.

Q7. Compare RSA and ECC as asymmetric cryptography algorithms. Which is more suitable for blockchain and why?

ANSWER

1. Fundamental Basis

RSA: Based on INTEGER FACTORIZATION problem - hard to factor the product of two large prime numbers.

ECC: Based on ELLIPTIC CURVE DISCRETE LOGARITHM PROBLEM (ECDLP) - hard to find d given T = dP on an elliptic curve over a finite field.

2. Comprehensive Comparison

Feature	RSA	ECC
Underlying Problem	Integer factorization of large primes	Discrete logarithm on elliptic curves (ECDLP)
Invented By	Rivest, Shamir, Adelman (1977)	Koblitz and Miller (1985)
Key Size (Equivalent Security)	3,072 bits for high security	256 bits for high security (12x smaller)
Speed	Slower - computationally intensive	Faster - operations on curve are efficient
Memory Requirements	Higher - larger keys need more storage	Lower - smaller keys require less memory
Suitability for Embedded/Mobile	Poor - large keys impractical	Excellent - small keys ideal for IoT, mobile
Digital Signatures	RSA Signatures	ECDSA (Elliptic Curve DSA)
Key Exchange	RSA Key Exchange	ECDH (Elliptic Curve Diffie-Hellman)
Blockchain Use	Rarely used in blockchain	Primary algorithm in Bitcoin (secp256k1) and Ethereum
Patent Status	RSA patent expired in 2000	Some ECC patents existed (mostly expired)
Implementation Complexity	Relatively straightforward	More complex due to curve operations

3. Why ECC is More Suitable for Blockchain

- SMALLER KEY SIZE: 256-bit ECC = security of 3,072-bit RSA - critical for blockchain where storage efficiency matters

- FASTER OPERATIONS: ECC operations are computationally faster than RSA for the same security level
- REDUCED TRANSACTION SIZE: Smaller keys mean smaller digital signatures - reduces blockchain data/storage overhead
- MOBILE AND EMBEDDED COMPATIBILITY: Wallets often run on mobile devices - ECC's efficiency is crucial
- ESTABLISHED STANDARD: ECDSA over secp256k1 curve is the standard for Bitcoin, Ethereum, and most major blockchains
- BATTERY EFFICIENCY: Less computation = less battery usage on mobile wallets

4. ECC in Blockchain Specifically

- Bitcoin: Uses ECDSA with the secp256k1 curve for signing all transactions
- Ethereum: Also uses ECDSA (same secp256k1 curve) for transaction signing and address generation
- Private key: A 256-bit random number selected from the secp256k1 curve's field
- Public key: Derived by multiplying the generator point G by the private key d: Public_Key = d * G
- Bitcoin address: Derived from public key via SHA-256 then RIPEMD-160 hashing

EXAM TIP: The critical comparison point: ECC 256-bit = RSA 3,072-bit security. ECC is THE algorithm used in blockchain (Bitcoin and Ethereum use ECDSA over secp256k1). Always mention the blockchain applications: ECDSA for signing transactions and ECDH for key exchange. The secp256k1 curve name earns extra marks in blockchain-specific questions.

Q8. Explain Digital Signatures. How do they work in public key cryptography? What security services do they provide? Discuss ECDSA in the context of blockchain.

ANSWER

1. Definition of Digital Signatures

Digital Signature: A cryptographic mechanism that uses a sender's PRIVATE KEY to sign a message, and the corresponding PUBLIC KEY to verify the signature. Provides authentication, integrity, and non-repudiation.

Digital signatures are NOT the same as encryption. They provide message authentication and integrity - not confidentiality.

2. How Digital Signatures Work

Signing Process (Sender's Side)

- Sender takes the plaintext message P
- Sender applies the SIGNING FUNCTION S using their PRIVATE KEY
- Produces signed data C (the message with the digital signature)

```
C = S(Sender_PrivateKey, P)
```

- C is transmitted over the channel to the receiver

Verification Process (Receiver's Side)

- Receiver receives signed data C
- Receiver applies the VERIFICATION FUNCTION V using the SENDER'S PUBLIC KEY
- Verifies that C was indeed produced by the sender (not an impostor)

```
P = V(Sender_PublicKey, C)
```

- If verification succeeds: message is authentic, integrity confirmed, origin authenticated

3. Security Services Provided by Digital Signatures

Security Service	How Signatures Provide It
Authentication (Data Origin)	Verifying signature with sender's public key proves message came from the private key holder
Integrity	Any modification to the message invalidates the signature
Non-Repudiation	Sender cannot deny signing (only they have the private key that created the signature)
Identification	Combined with challenge-response protocols, identifies the entity in a transaction

NOTE: Digital signatures do NOT provide confidentiality (the message is not encrypted in the signing model). For confidentiality, encryption with the receiver's public key is used separately.

4. ECDSA - Elliptic Curve Digital Signature Algorithm

ECDSA: The most widely used digital signature scheme in blockchain. Based on the Elliptic Curve Discrete Logarithm Problem (ECDLP). Used in both Bitcoin and Ethereum.

ECDSA works with:

- A private key d (a 256-bit random number on the secp256k1 curve)
- A public key Q = d * G (where G is the generator point of the secp256k1 curve)
- The secp256k1 curve: y^2 = x^3 + 7 (where a=0, b=7) over a prime finite field

ECDSA in Blockchain Transactions

- Every transaction in Bitcoin/Ethereum is signed with the sender's ECDSA private key
- This proves the transaction was authorized by the rightful owner of the funds
- Nodes on the network VERIFY the signature using the sender's public key before accepting the transaction
- No private key = cannot sign transactions = cannot spend funds
- This is why losing a private key means permanent loss of access to blockchain funds

5. Complete Picture - Public Key Crypto Mechanisms in Blockchain

Mechanism	Algorithm	Use in Blockchain
Key Generation	ECDSA (secp256k1)	Private key randomly generated; public key derived via point multiplication
Transaction Signing	ECDSA	Owner signs transaction with private key to authorize spending
Signature Verification	ECDSA	Network nodes verify signature using sender's public key
Key Exchange	ECDH	Establishing shared secrets for encrypted communication
Address Generation	ECDSA + SHA-256 + RIPEMD-160	Bitcoin: Hash public key to derive wallet address

EXAM TIP: Always distinguish signing from encryption: Signing uses PRIVATE key; Verification uses PUBLIC key. Encryption uses RECEIVER'S public key; Decryption uses RECEIVER'S private key. Digital signatures provide: Authentication + Integrity + Non-repudiation (but NOT confidentiality). For blockchain questions, always mention ECDSA with secp256k1 curve - both Bitcoin and Ethereum use this.

QUICK REFERENCE: Key Terms and One-Line Definitions

Term	One-Line Definition
Modular Arithmetic	Clock arithmetic where numbers wrap around at a fixed modulus; deals with remainders
Finite Field (Galois Field)	Field with a finite set of elements; used in ECC for accurate error-free arithmetic
Prime Field	Finite field with prime number of elements; operations done modulo a prime p
Group	Set with 4 axioms: closure, associativity, identity element, inverse element
Abelian Group	Group + commutativity (5 axioms); A*B = B*A for all elements
Ring	Abelian group with more than one defined operation; closure + associative + distributive
Cyclic Group	Group generated by a single element called the generator; essential for discrete log problem
Order (Cardinality)	The number of elements in a field
Asymmetric Cryptography	Encryption key (public) differs from decryption key (private); also called public key cryptography
RSA	Public key algorithm based on integer factorization; invented 1977 by Rivest, Shamir, Adelman
RSA Encryption	C = P^e mod n (plaintext raised to power e, modulo n)
RSA Decryption	P = C^d mod n (ciphertext raised to power d, modulo n)
RSA Public Key	The pair (n, e) where n = p*q and e is co-prime of (p-1)(q-1)
RSA Private Key	d = modular inverse of e; calculated using Extended Euclidean Algorithm
Integer Factorization	Hard problem: easy to multiply two large primes; hard to factor the result (basis of RSA)
Discrete Logarithm	Hard problem: easy to compute g^x mod n; hard to find x given the result
ECC	Public key crypto based on discrete logarithm problem on elliptic curves over finite fields
ECC Advantage	256-bit ECC = 3,072-bit RSA security; smaller key, faster operations
Elliptic Curve Equation	y^2 = x^3 + Ax + B (Weierstrass equation); non-singular curve over finite field
Non-Singular Condition	4a^3 + 27b^2 != 0 (mod p); ensures curve has no repeated roots/cusps
Point of Infinity	Special value on elliptic curve; provides identity element for group operations
Point Addition (P+Q)	Two different points added; secant line drawn; third intersection point mirrored to get R
Point Doubling (2P)	Point added to itself; tangent line drawn at P; intersection mirrored to get 2P
Point Multiplication (dP)	Scalar multiplication; P added to itself d times; computed via double-and-add algorithm

Double-and-Add Algorithm	Efficient method: convert d to binary; for each bit do double (+ add if bit=1); exponential speedup
ECDLP	Elliptic Curve Discrete Log Problem: given T=dP, finding d is computationally infeasible
ECDSA	Elliptic Curve Digital Signature Algorithm; used in Bitcoin and Ethereum for transaction signing
ECDH	Elliptic Curve Diffie-Hellman; used for key exchange in ECC-based systems
secp256k1	The specific ECC curve used by Bitcoin and Ethereum: y^2 = x^3 + 7 over prime field
Digital Signature	Private key signs; public key verifies; provides authentication, integrity, non-repudiation
Key Establishment	Protocols to set up keys over an insecure channel (e.g., Diffie-Hellman)
Extended Euclidean Algorithm	Algorithm used to compute RSA private key d from p, q, and e
Co-prime	e and (p-1)(q-1) are co-prime if their only common divisor is 1; gcd(e,(p-1)(q-1))=1

MUST-KNOW POINTS FOR EXAM

1. Asymmetric cryptography: ENCRYPTION uses receiver's PUBLIC key; DECRYPTION uses receiver's PRIVATE key. SIGNING uses sender's PRIVATE key; VERIFICATION uses sender's PUBLIC key.

2. RSA is based on INTEGER FACTORIZATION. Key generation: (1) n=p*q, (2) choose e co-prime to (p-1)(q-1), (3) public key=(n,e), (4) private key d: e*d=1 mod(p-1)(q-1). Encryption: C=P^e mod n. Decryption: P=C^d mod n.

3. ECC is based on the ELLIPTIC CURVE DISCRETE LOGARITHM PROBLEM. Main advantage: 256-bit ECC = 3,072-bit RSA security. Used in Bitcoin and Ethereum (ECDSA over secp256k1 curve).

4. Elliptic curve equation: y^2 = x^3 + Ax + B. Non-singular condition: 4a^3 + 27b^2 != 0 mod p. Operations performed over PRIME FINITE FIELDS for crypto.

5. Group axioms (4): Closure + Associativity + Identity element + Inverse element. Abelian group adds a 5th: Commutativity (A*B = B*A).

6. Point Addition (P+Q): uses a SECANT line through P and Q. Point Doubling (2P): uses a TANGENT line at P. Both operations result in another point on the curve (mirroring the intersection).

7. Double-and-Add Algorithm: Convert d to binary. Starting from MSB (skip it), for each bit: if 1 then DOUBLE+ADD; if 0 then DOUBLE only. Example: 9P with 9=1001 binary needs only 3 doubles + 1 add.

8. Public key algorithms are SLOWER than symmetric algorithms. Used for KEY EXCHANGE and DIGITAL SIGNATURES only - NOT for bulk data encryption. Hybrid systems use asymmetric for key exchange + symmetric for data.

9. RSA security: 1,024-bit is NO LONGER SECURE. Minimum 2,048-bit recommended. Security depends on difficulty of factoring n = p*q back to p and q.

10. Digital signatures provide: Authentication + Integrity + Non-Repudiation. They do NOT provide Confidentiality. This is a critical distinction.

11. ECDSA signing: every blockchain transaction is signed with owner's PRIVATE key. Nodes verify using PUBLIC key. No private key = cannot spend funds = permanent loss if key is lost.

12. Cyclic groups are generated by a single GENERATOR element. The discrete logarithm problem: easy to compute g^x, hard to find x from result. This one-way property is the basis of ECC security.

13. Modular arithmetic is clock arithmetic: 50 mod 11 = 6 (remainder after division). Foundation of both RSA (integer factorization mod n) and discrete logarithm problems.

14. Security mechanisms of public key cryptosystems: Key Establishment + Digital Signatures + Identification + Encryption + Decryption. Non-repudiation is provided ONLY via digital signatures.

Q1. Explain the Elliptic Curve Discrete Logarithm Problem (ECDLP). What is the role of domain parameters in ECC, and how does ECDLP form the basis of ECC security?

ANSWER

1. Introduction to ECDLP

> **ECDLP (Elliptic Curve Discrete Logarithm Problem):** The computationally hard problem that forms the security foundation of ECC. Based on the principle that, under certain conditions, all points on an elliptic curve form a cyclic group, and finding the scalar multiplier from the result of scalar multiplication is computationally infeasible.

Key insight: Even if we know points P and T on an elliptic curve, it is computationally infeasible to reconstruct the sequence of all double-and-add operations used to compute d. This is the one-way function on which ECDLP is built.

2. Formal Definition of ECDLP

Consider an elliptic curve E with two elements P (generator point) and T (public key). The ECDLP is to find the integer d, where:

```
1 <= d <= #E  such that:  P + P + ... + P = dP = T
```

Variable	Meaning	Public or Private?
P	The generator point (starting/base point) on the elliptic curve	PUBLIC - known to everyone
T	The resulting point after scalar multiplication; this is the public key (x, y)	PUBLIC - published openly
d	The integer scalar (private key) used to generate T from P	PRIVATE - must be kept secret
#E	The ORDER of the elliptic curve (number of points in the cyclic group)	PUBLIC - defined by curve parameters
dP	Scalar multiplication: P added to itself d times	Easy to compute forward

Critical one-way property: Given d and P, computing T = dP is EASY (double-and-add algorithm). But given T and P, finding d is computationally INFEASIBLE for large enough curve orders.

A cyclic group is formed by a combination of ALL POINTS on the elliptic curve and the POINT OF INFINITY. The order #E determines the security level - larger order means higher security.

3. ECC Domain Parameters

> **Domain Parameters:** A set of parameters that fully define an elliptic curve and its associated group. A key pair is always linked with specific domain parameters of an elliptic curve.

Domain parameters are defined as a sextuple T = (p, a, b, G, n, h):

Parameter	Symbol	Description
Field Size	p	The prime number that specifies the size of the finite field; determines the field Fp over which the curve is defined
Curve Coefficients	a, b	The two elements from the field that define the elliptic curve equation y^2 = x^3 + ax + b
Generator / Base Point	G = (Xg, Yg)	The base point that generates the required cyclic subgroup; also called the generator point. Can be in compressed or uncompressed form.
Order of Subgroup	n	The order of the generator point G; the number of points in the cyclic subgroup generated by G
Cofactor	h = #E(Fq)/n	The cofactor of the subgroup; ratio of total curve points to subgroup order; usually h=1 for secure curves

4. Why ECDLP is Computationally Hard

- For large finite fields (256-bit prime p), the number of points on the curve is astronomically large
- The double-and-add algorithm makes computing dP efficient (forward), but no efficient algorithm exists for the inverse
- The best known algorithms for solving ECDLP run in exponential time for properly chosen curves
- This hardness is what makes ECC private keys secure: even knowing the public key T and generator P, d cannot be recovered

NOTE: ECDLP vs Discrete Logarithm Problem: Both are one-way function based hard problems. The DLP in modular arithmetic (used in Diffie-Hellman and DSA) is: given g^x mod p = y, find x. ECDLP replaces modular exponentiation with elliptic curve point multiplication - this allows the same security with much smaller key sizes.

EXAM TIP: The ECDLP statement must include: (1) the formal equation dP = T, (2) explanation of P (generator), T (public key), and d (private key), (3) the one-way property: easy forward (dP), hard reverse (find d from T and P). Always mention that #E is the order of the elliptic curve. The sextuple T=(p,a,b,G,n,h) is often directly asked in exams.

Q2. Explain the SECP256K1 curve used in Bitcoin. What are its domain parameters, and what makes it suitable for blockchain?

ANSWER

1. What is SECP256K1?

SECP256K1: A specific standardized elliptic curve defined by the Standards for Efficient Cryptography Group (SECG). Used in Bitcoin and Ethereum for key generation and transaction signing via ECDSA. It is a Koblitz curve over a 256-bit prime field.

SECP256K1 belongs to the secp family of curves. The name means: SEC (Standards for Efficient Cryptography) + P (prime field) + 256 (256-bit prime) + K (Koblitz curve) + 1 (first in the series).

2. SECP256K1 Domain Parameters Sextuple T = (p, a, b, G, n, h)

A. Prime p (Field Size)

The prime p defines the size of the finite field Fp:

```
p = 2^256 - 2^32 - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1
```

- This is a 256-bit prime number (approximately 1.16 x 10^77)
- Chosen for efficiency: the special form allows fast modular arithmetic

B. Curve Coefficients a and b

```
a = 0    (no x term)
b = 7    (constant term)
```

This gives the curve equation:

```
y^2 = x^3 + 7   (over Fp)
```

- The simplest possible Weierstrass curve (a=0 eliminates the Ax term entirely)
- This is why secp256k1 is sometimes called the 'y^2 = x^3 + 7' curve

C. Generator Point G (Base Point)

- G is the starting base point that generates the entire cyclic subgroup
- Can be represented in COMPRESSED form (02 prefix + x-coordinate only) or UNCOMPRESSED form (04 prefix + both x and y coordinates)
- Compressed form works because given x and the least significant bit of y, the full point can be recovered from the curve equation
- Using compressed form halves the storage needed for public keys

D. Order n

- n is the order of the generator point G (size of the cyclic subgroup)
- For secp256k1, n is a large 256-bit prime number, approximately the same size as p
- The private key d must satisfy: 0 < d < n

E. Cofactor h

```
h = 1
```

- The cofactor h=1 means the entire curve group IS the cyclic subgroup generated by G
- A cofactor of 1 is ideal for security - all points on the curve are valid

3. OpenSSL Verification of SECP256K1 Parameters

The following OpenSSL command displays SECP256K1 domain parameters:

```
$ openssl ecparam -param_enc explicit -text -noout -name secp256k1  Field
Type: prime-field Prime: FFFFFFFF...FFFFFFFC2F  (= 2^256 - 2^32 - 977) A: 0 B:
7 (0x7) Generator (uncompressed): 04:79:BE:66:7E... (65 bytes) Order:
FFFFFFFF...BAAEDCE6AF48A03BBFD25E8CD0364141 Cofactor: 1 (0x1)
```

4. Why SECP256K1 is Ideal for Blockchain

Feature	Benefit for Blockchain
256-bit prime field	Provides approximately 128-bit security level - computationally infeasible to break
Koblitz curve (a=0)	Allows faster computation using special algorithms; optimized for performance
h = 1 (cofactor)	All curve points form one group; no small subgroup attacks possible
Standardized parameters	Widely reviewed and trusted; no hidden backdoors (unlike some NIST curves)
Compressed public keys	32-byte private key, 33-byte compressed public key; efficient storage in blockchain

Small key size	256-bit keys provide same security as 3,072-bit RSA; reduces transaction size

EXAM TIP: The secp256k1 equation y^2 = x^3 + 7 (because a=0, b=7) is very commonly tested. Memorize: p is a 256-bit prime of the special form 2^256 - 2^32 - 977, a=0, b=7, h=1. Know the sextuple (p, a, b, G, n, h). The compressed vs uncompressed generator distinction (02 prefix vs 04 prefix) earns extra marks. Bitcoin AND Ethereum both use secp256k1.

Q3. Explain how RSA key generation, encryption, and decryption are performed using OpenSSL. What are the key components visible in an RSA private key?

ANSWER

1. RSA Key Generation Using OpenSSL

Step 1: Generate the RSA Private Key

Command to generate a 1024-bit RSA private key (stored in PEM format):

```
$ openssl genpkey -algorithm RSA -out privatekey.pem -pkeyopt
rsa_keygen_bits:1024
```

- Output: A file named privatekey.pem containing the private key in Base64-encoded PEM format
- Format: Enclosed between -----BEGIN PRIVATE KEY----- and -----END PRIVATE KEY-----
- For production systems, 2048-bit minimum is recommended (1024-bit is no longer secure)

Step 2: Derive the Public Key from the Private Key

Since the public key is mathematically linked to the private key, it can be derived:

```
$ openssl rsa -pubout -in privatekey.pem -out publickey.pem
```

- Output: A file named publickey.pem containing the public key
- Format: Enclosed between -----BEGIN PUBLIC KEY----- and -----END PUBLIC KEY-----
- The public key can be freely shared with anyone who wants to send encrypted messages

2. Components of an RSA Private Key

Using the following command reveals all internal RSA key components:

```
$ openssl rsa -text -in privatekey.pem
```

Component	Description	Role in RSA
modulus (n)	The product of two large prime numbers p and q; n = p * q	Part of both public and private key; used in C=P^e mod n and P=C^d mod n
publicExponent (e)	Common value: 65537 (0x10001); co-prime to (p-1)(q-1)	Part of public key (n, e); used in encryption: C = P^e mod n
privateExponent (d)	Modular inverse of e; computed via Extended Euclidean Algorithm	Private key; used in decryption: P = C^d mod n
prime1 (p)	First large prime number	Must be kept secret; used to compute private key d
prime2 (q)	Second large prime number	Must be kept secret; used to compute private key d

NOTE: The public exponent is almost always 65537 (= 2^16 + 1). This specific value is chosen because it is a Fermat prime, making modular exponentiation efficient while still being large enough for security. The exponent 3 is sometimes used but is considered less secure.

3. RSA Encryption Using OpenSSL

First create a plaintext file, then encrypt it using the public key:

```
$ echo datatoencrypt > message.txt $ openssl rsautl -encrypt -inkey
publickey.pem -pubin -in message.txt -out message.rsa
```

- Output: message.rsa - a binary file containing the RSA-encrypted data
- The encrypted file appears as scrambled binary data when opened in a text editor
- Encryption uses the RECEIVER'S PUBLIC KEY (-pubin flag specifies input is a public key)

4. RSA Decryption Using OpenSSL

Decrypt the RSA-encrypted file using the private key:

```
$ openssl rsautl -decrypt -inkey privatekey.pem -in message.rsa -out
message.dec $ cat message.dec datatoencrypt
```

- Decryption uses the RECEIVER'S PRIVATE KEY
- The original plaintext 'datatoencrypt' is successfully recovered
- Only the holder of the corresponding private key can decrypt

5. Key Security Reminders

- In production systems, safeguarding the private key is of UTMOST IMPORTANCE
- The private key must NEVER be shared or exposed
- Public and private keys are stored in Base64-encoded PEM format
- PEM (Privacy Enhanced Mail) format uses Base64 encoding with header/footer lines

EXAM TIP: The OpenSSL commands are frequently asked in practical exam questions. Memorize: genpkey (generate private key), rsa -pubout (derive public key), rsautl -encrypt (encrypt with public key), rsautl -decrypt (decrypt with private key). Know the 5 components of RSA private key: modulus, publicExponent (65537), privateExponent, prime1, prime2. Always state that encryption uses public key and decryption uses private key.

Q4. Explain how ECC key generation and curve parameter exploration are performed using OpenSSL with the SECP256K1 curve.

ANSWER

1. Listing Available ECC Curves

OpenSSL supports a large number of standardized ECC curves. All available curves can be listed:

```
$ openssl ecparam -list_curves
```

Sample output includes curves like:

- secp256k1: SECG curve over a 256-bit prime field (used in Bitcoin/Ethereum)
- secp384r1: NIST/SECG curve over a 384-bit prime field

- secp521r1: NIST/SECG curve over a 521-bit prime field
- prime192v1: NIST/X9.62/SECG curve over a 192-bit prime field
- brainpoolP384r1: RFC 5639 curve over a 384-bit prime field

Note: Different types of curves must be chosen carefully to ensure cryptographic security guarantees. NIST curves are published in the FIPS 186 document. Safe curve selection criteria are available at safecurves.cr.yp.to.

2. Generating an ECC Private Key (SECP256K1)

```
$ openssl ecparam -name secp256k1 -genkey -noout -out ec-privatekey.pem
```

- Output: ec-privatekey.pem containing the ECC private key
- Format: Between -----BEGIN EC PRIVATE KEY----- and -----END EC PRIVATE KEY-----
- The private key is a 256-bit integer on the secp256k1 curve

3. Deriving the ECC Public Key

```
$ openssl ec -in ec-privatekey.pem -pubout -out ec-pubkey.pem $ cat ec-
pubkey.pem -----BEGIN PUBLIC KEY----- MFYwEAYHKoZIzj0CAQYFK4EEAAoDQgAE... ----
-END PUBLIC KEY-----
```

- The public key is derived from the private key via scalar multiplication: Q = d * G
- Output: ec-pubkey.pem containing the ECC public key

4. Exploring ECC Key Details

The private key components can be examined in detail:

```
$ openssl ec -in ec-privatekey.pem -text -noout  Private-Key: (256 bit) priv:
00:91:d4:22:6f:4d:64:08:1f...(32 bytes) pub: 04:d0:6d:f7:98:26:78:3c:a6...(65
bytes) ASN1 OID: secp256k1
```

Component	Description
priv (32 bytes)	The 256-bit private key d - a randomly chosen integer; must be kept secret
pub (65 bytes)	The uncompressed public key point (x, y) on the curve; starts with 04 prefix for uncompressed form
ASN1 OID	The Object Identifier confirming which curve is being used (secp256k1 in this case)

5. Exploring SECP256K1 Curve Parameters

Generate a file with secp256k1 parameters and analyze them:

```
$ openssl ecparam -name secp256k1 -out secp256k1.pem $ openssl ecparam -in
secp256k1.pem -text -param_enc explicit -noout  Field Type: prime-field Prime:
FFFFFFFF...FFFFFFFC2F A: 0 B: 7 (0x7) Generator (uncompressed):
04:79:BE:66:7E... Order: FFFFFFFF...BAAEDCE6... Cofactor: 1 (0x1)
```

- A=0 and B=7 confirm the curve equation: y^2 = x^3 + 7
- The generator G is shown in uncompressed form (04 prefix + x + y coordinates = 65 bytes)
- Order n is approximately the same size as prime p (both 256-bit)
- Cofactor h=1 confirms the entire curve group is the cyclic subgroup

6. Compressed vs Uncompressed Public Key Forms

Form	Prefix	Size	Contents	Use Case

Uncompressed	04	65 bytes	04 + x-coordinate (32 bytes) + y-coordinate (32 bytes)	Older implementations; provides both coordinates
Compressed	02 or 03	33 bytes	02/03 + x-coordinate (32 bytes) only	Modern standard; 02 if y is even, 03 if y is odd

Compressed keys work because: given x and the least significant bit (parity) of y, the full y-coordinate can be recovered from the curve equation. This halves the size of stored public keys.

EXAM TIP: The key OpenSSL commands: ecparam -list_curves (list curves), ecparam -name secp256k1 -genkey (generate private key), ec -pubout (derive public key), ec -text -noout (view key details). The private key = 32 bytes (256-bit integer); uncompressed public key = 65 bytes (04 + 32 + 32). Compressed public key = 33 bytes. Always note that public key is derived via Q = d * G.

Q5. Explain Digital Signatures in detail. Describe the signing process, properties, approaches (Sign-then-Encrypt vs Encrypt-then-Sign), and the RSA digital signature scheme.

ANSWER

1. Definition and Purpose

Digital Signature: A cryptographic mechanism that provides a means of associating a message with the entity from which it originated. Used to provide DATA ORIGIN AUTHENTICATION and NON-REPUDIATION.

In blockchain: Transactions are digitally signed by senders using their private key before broadcasting. This proves the sender is the rightful owner of the asset. Other nodes verify the signature to confirm ownership.

The operation of a generic digital signature function is shown in the following diagram:

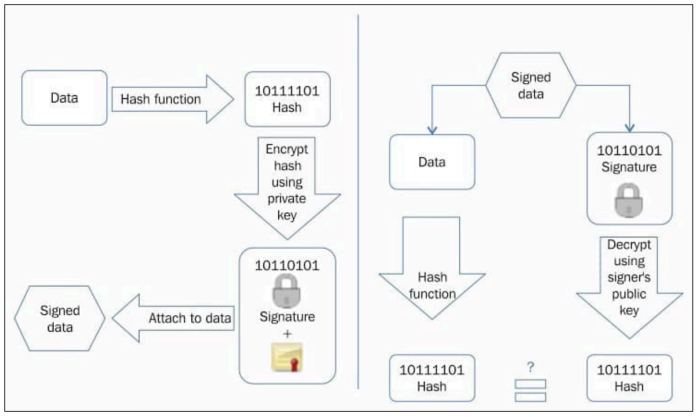

Figure 4.10: Digital signing (left) and verification process (right) (example of RSA digital signatures)

2. The RSA Digital Signature Process (2 Steps)

The fundamental idea is to first compute the hash of the data, then sign the hash with the private key:

Step 1: Hash the Data

- Calculate the hash value of the data packet using a hash function (e.g., SHA-256)
- This provides DATA INTEGRITY guarantee - the hash can be recomputed at receiver's end and compared

- Note: Message signing can technically work without hashing first, but that is NOT considered secure

Step 2: Sign the Hash

- Sign the hash value with the SIGNER'S PRIVATE KEY
- As only the signer has the private key, AUTHENTICITY of the signature is ensured
- The signature is attached to the original data and sent to the receiver

Verification Process at Receiver

- Receiver receives the signed data (original data + signature)
- Receiver decrypts the signature using the SENDER'S PUBLIC KEY to recover the original hash
- Receiver independently recomputes the hash of the received data
- If both hashes MATCH: signature is valid, data is authentic and unmodified
- If hashes DO NOT match: data has been tampered with or signature is forged

3. Properties of Digital Signatures

Property	Definition	Implication
Authenticity	Digital signatures are verifiable by the receiving party	Receiver can confirm the message came from the legitimate sender
Unforgeability (Non-Repudiation)	Only the sender can use the private key for signing; adversaries cannot fabricate a valid signature without the private key	Sender cannot deny having signed the message - provides non-repudiation
Non-Reusability	A digital signature cannot be separated from its message and reused for another message; the signature is firmly bound to its corresponding message	Prevents copy-paste signature attacks; signature is message-specific

4. Two Approaches: Sign-then-Encrypt vs Encrypt-then-Sign

Approach 1: Sign-then-Encrypt (MORE SECURE)

- Sender SIGNS the data using their private key
- Appends the signature to the data
- ENCRYPTS both the data AND the signature using the RECEIVER'S PUBLIC KEY
- This is considered the MORE SECURE scheme

Process: Data -> Sign with Sender's Private Key -> Append Signature -> Encrypt (Data+Signature) with Receiver's Public Key -> Send

Approach 2: Encrypt-then-Sign (LESS SECURE)

- Sender ENCRYPTS the data using the receiver's public key first
- Then SIGNS the encrypted data using sender's private key

Process: Data -> Encrypt with Receiver's Public Key -> Sign encrypted data with Sender's Private Key -> Send

Why Sign-then-Encrypt is more secure: In Encrypt-then-Sign, the signature is applied to the encrypted data, not the original data. An adversary could potentially strip the signature and re-sign with their own key, impersonating the sender. Sign-then-Encrypt binds the signature to the original plaintext inside the encrypted envelope.

5. Digital Signature Schemes

- RSA-based digital signatures: Use RSA private key for signing; RSA public key for verification
- DSA (Digital Signature Algorithm): Based on modular exponentiation and discrete logarithm problem

- ECDSA (Elliptic Curve DSA): DSA applied to elliptic curves; used in Bitcoin and Ethereum

RSA is the most commonly used scheme; however, ECDSA-based schemes are becoming popular due to ECC's advantages: same security with smaller key sizes and faster key generation.

6. RSA Key Size vs ECC Key Size for Digital Signatures

RSA Key Size (bits)	Equivalent ECC Key Size (bits)	Security Level
1,024	160	Low (legacy only)
2,048	224	Standard
3,072	256	High
7,680	384	Very High
15,360	521	Ultra High

NOTE: In practice, digital certificates that contain digital signatures are issued by a Certificate Authority (CA) that associates a public key with an identity. This solves the authenticity problem of public keys - the CA vouches that a given public key belongs to the claimed identity.

EXAM TIP: The 3 properties of digital signatures (Authenticity, Unforgeability/Non-Repudiation, Non-Reusability) are frequently asked directly. Always explain the 2-step signing process: (1) Hash the data, (2) Sign the hash with private key. For Sign-then-Encrypt vs Encrypt-then-Sign, clearly state that Sign-then-Encrypt is MORE SECURE and explain why. The RSA/ECC key equivalence table is often directly tested.

Q6. Explain the Elliptic Curve Digital Signature Algorithm (ECDSA). Describe key generation, the signing process, and verification. How is ECDSA used in blockchain?

ANSWER

1. Definition of ECDSA

ECDSA: A Digital Signature Algorithm (DSA) based on elliptic curves. Based on modular exponentiation and the discrete logarithm problem applied to elliptic curves. Used on Bitcoin and Ethereum blockchain platforms to validate messages and provide data integrity services.

ECDSA = Elliptic Curve + DSA. The DSA is a standard for digital signatures based on the discrete logarithm problem. ECDSA applies this to the more efficient elliptic curve group operations.

2. ECDSA Key Generation

Step 1: Define the elliptic curve E with the following parameters:

- Modulus P (prime field size)
- Coefficients a and b (curve equation parameters)
- Generator point A (also called G in secp256k1) that forms a cyclic group of PRIME ORDER q

Step 2: Choose the private key:

```
Choose integer d randomly such that:  0 < d < q
```

Step 3: Calculate the public key:

```
B = d * A   (scalar multiplication of generator A by private key d)
```

Key pair representations:

```
Public Key:  Kpb = (p, a, b, q, A, B)    [sextuple]
Private Key: Kpr = d                      [integer scalar]
```

3. Why ECDSA is Used in Blockchain

Advantage	Explanation
Same security as RSA with smaller keys	256-bit ECDSA = 3,072-bit RSA; reduces transaction and block size in blockchain
Faster key generation	ECC key generation is significantly faster than RSA for the same security level
Faster signing and verification	Elliptic curve operations are computationally lighter; critical for high-throughput blockchain networks
Non-repudiation	Provides mathematically proven non-repudiation for blockchain transactions
Smaller signatures	Smaller signature sizes mean less data per transaction; reduces blockchain bloat
secp256k1 compatibility	Both Bitcoin and Ethereum use secp256k1 curve; standard is well-established and reviewed

4. ECDSA in Bitcoin Transactions

- Every Bitcoin transaction output is locked to a public key (or address derived from it)
- To spend the output, the owner signs the transaction with their ECDSA private key
- Other nodes verify the signature using the owner's public key
- If verification succeeds: transaction is valid; the spender owns the funds
- If verification fails: transaction is rejected; funds cannot be spent

Key derivation chain in Bitcoin (using secp256k1):

```
Random integer d  -->  Public Key Q = d * G  -->  Hash (SHA-256 +
RIPEMD-160)  -->  Bitcoin Address
```

5. ECDSA vs RSA for Digital Signatures

Feature	RSA Signatures	ECDSA
Underlying Problem	Integer Factorization	Elliptic Curve Discrete Logarithm (ECDLP)
Key Size (equivalent security)	3,072 bits for 128-bit security	256 bits for 128-bit security
Speed	Slower - large modular exponentiations	Faster - elliptic curve operations
Signature Size	Larger (same as key size)	Smaller (approximately twice the key size)
Use in Blockchain	Rarely; occasionally in some protocols	Standard for Bitcoin, Ethereum, and most blockchains
Standard	PKCS#1, RFC 3447	ANSI X9.62, SEC1, RFC 6979

EXAM TIP: ECDSA key generation: (1) Define curve E with parameters, (2) Choose random d (private key) where 0 < d < q, (3) Compute B = dA (public key). The public key is the SEXTUPLE (p, a, b, q, A, B); the

private key is just the INTEGER d. Always mention Bitcoin and Ethereum use ECDSA over secp256k1. ECDSA provides non-repudiation, integrity, and authentication for blockchain transactions.

Q7. Compare RSA and ECC comprehensively for both encryption and digital signatures. Discuss their suitability for blockchain applications with a focus on key sizes and performance.

ANSWER

1. Overview

Both RSA and ECC are asymmetric (public key) cryptography schemes. They differ in their underlying mathematical problems, key sizes, performance characteristics, and suitability for different applications.

2. Mathematical Foundation Comparison

Aspect	RSA	ECC (including ECDSA)
Hard Problem	Integer Factorization: hard to factor n = p*q	ECDLP: given T = dP, hard to find d
Mathematical Domain	Modular arithmetic over integers	Group theory on elliptic curves over finite fields
Key Relationship	n = p*q; d = inverse of e mod (p-1)(q-1)	Public key B = d*A (scalar point multiplication)
One-Way Function	Multiplication easy; factoring hard	dP easy to compute; finding d from T and P hard
Invented	1977 by Rivest, Shamir, Adelman	1985 by Koblitz and Miller

3. Key Size Equivalence Table

Security Level (bits)	RSA Key Size	ECC Key Size	Ratio (RSA:ECC)
80-bit security	1,024 bits	160 bits	6.4:1
112-bit security	2,048 bits	224 bits	9.1:1
128-bit security	3,072 bits	256 bits	12:1
192-bit security	7,680 bits	384 bits	20:1
256-bit security	15,360 bits	521 bits	29.5:1

4. Performance Comparison

Operation	RSA	ECC	Winner
Key Generation	Slow (requires large prime generation and factoring)	Fast (simple random number generation + point multiplication)	ECC

Signing/Encryption	Slow (large modular exponentiation)	Fast (elliptic curve point operations)	ECC
Verification/Decryption	Fast (small public exponent e=65537)	Moderate	RSA (for verification)
Storage Requirements	Large (2048-3072 bit keys)	Small (256-384 bit keys)	ECC
Bandwidth	High (large keys, large signatures)	Low (small keys, compact signatures)	ECC
Implementation Complexity	Simple and well-understood	More complex (curve arithmetic)	RSA

5. Suitability for Blockchain

Criterion	RSA	ECC / ECDSA	Blockchain Preference
Transaction size	Large signatures increase block size	Compact signatures minimize block data	ECC preferred
Processing speed	Slower; performance bottleneck	Faster operations; higher throughput	ECC preferred
Mobile wallet support	Heavy; poor for resource-constrained devices	Lightweight; ideal for mobile/IoT wallets	ECC preferred
Storage efficiency	Large keys waste blockchain storage	Small keys conserve storage	ECC preferred
Security for key size	Needs 3,072+ bits for adequate security	256 bits provides equivalent security	ECC preferred
Industry adoption	Used in legacy PKI systems	Standard for Bitcoin, Ethereum, all major blockchains	ECC preferred

Conclusion: ECC (specifically ECDSA with secp256k1) is overwhelmingly preferred for blockchain because it provides equivalent security with 12x smaller keys, faster operations, compact signatures, and lower resource requirements - all critical for a system where millions of transactions are processed and stored permanently.

EXAM TIP: The key size table (RSA 3,072 = ECC 256) is the most tested comparison point. For blockchain, always conclude that ECC/ECDSA is preferred. The 5 reasons ECC wins for blockchain: (1) smaller key size, (2) faster key generation, (3) compact signatures = smaller transactions, (4) mobile/IoT friendly, (5) both Bitcoin and Ethereum use secp256k1 ECDSA as their standard.

Q8. Describe the complete process of key generation and Bitcoin address derivation using ECC and cryptographic hash functions. Explain the role of SECP256K1 in this process.

ANSWER

1. Overview

Bitcoin uses a layered key derivation process combining ECC (secp256k1 curve) with multiple hash functions to derive wallet addresses from private keys. This process is one-way: you can go from private key to address easily, but cannot reverse the process.

2. Step-by-Step Key and Address Generation

Step 1: Private Key Generation

- Generate a cryptographically random 256-bit integer d
- Must satisfy: 0 < d < n (where n is the order of secp256k1 curve)
- This is the ECDSA private key - must be kept absolutely secret

```
Private Key d = random 256-bit integer (32 bytes)
```

Step 2: Public Key Derivation via Scalar Point Multiplication

- Multiply the generator point G of secp256k1 by the private key d
- This uses the double-and-add algorithm

```
Public Key Q = d * G  (scalar multiplication on secp256k1 curve)
```

- Result: Q is a point (x, y) on the secp256k1 curve
- Public key can be represented as uncompressed (04 + 32 bytes x + 32 bytes y = 65 bytes) or compressed (02/03 + 32 bytes x = 33 bytes)

Step 3: SHA-256 Hash of Public Key

```
SHA256_hash = SHA-256(Public Key Q)
```

- Apply SHA-256 hash function to the public key bytes
- Produces a 256-bit (32-byte) hash

Step 4: RIPEMD-160 Hash (Bitcoin Address Hash)

```
RIPEMD160_hash = RIPEMD-160(SHA256_hash)
```

- Apply RIPEMD-160 to the SHA-256 hash result
- Produces a 160-bit (20-byte) hash - this is the core of the Bitcoin address
- Combined operation known as HASH160: RIPEMD-160(SHA-256(public_key))

Step 5: Add Version Byte and Checksum

- Prepend a version byte (0x00 for mainnet)
- Compute a 4-byte checksum: first 4 bytes of SHA-256(SHA-256(versioned_hash))
- Append checksum to versioned hash

Step 6: Base58Check Encoding

- Encode the result using Base58Check encoding
- Produces the familiar Bitcoin address format (starts with '1' for mainnet P2PKH addresses)

```
Bitcoin Address = Base58Check(0x00 + RIPEMD160(SHA256(Public_Key)) + checksum)
```

3. Complete Flow Summary

Step	Input	Operation	Output	Size
1	Cryptographic RNG	Random number generation	Private Key d	256 bits (32 bytes)
2	Private Key d, Generator G	Scalar Point Multiplication: Q = d*G	Public Key Q (x,y)	65 bytes (uncompressed)
3	Public Key Q bytes	SHA-256 hash	SHA-256 hash	256 bits (32 bytes)

4	SHA-256 hash	RIPEMD-160 hash	RIPEMD-160 hash (address hash)	160 bits (20 bytes)
5	RIPEMD-160 hash	Add version byte + 4-byte checksum	Versioned hash with checksum	25 bytes
6	Versioned hash + checksum	Base58Check encoding	Bitcoin Address	~34 characters

4. Security of Each Step

- Private key -> Public key (Step 2): Protected by ECDLP - cannot reverse scalar multiplication
- Public key -> Address (Steps 3-4): Protected by pre-image resistance of SHA-256 and RIPEMD-160 - cannot find public key from address
- Two hash functions used (SHA-256 then RIPEMD-160): Double protection; different algorithms mean compromise of one doesn't break the scheme
- Checksum (Step 5): Detects typos in Bitcoin addresses - prevents accidental sending to wrong address

NOTE: Ethereum uses a different address derivation: it takes the Keccak-256 hash of the public key, then takes the last 20 bytes (40 hex characters) as the address. No RIPEMD-160, no Base58Check - Ethereum addresses are shown in hexadecimal with a '0x' prefix.

EXAM TIP: The complete derivation chain to memorize: Private Key (256-bit random) -> Public Key = d*G (secp256k1) -> SHA-256 -> RIPEMD-160 -> Add version byte + checksum -> Base58Check -> Bitcoin Address. RIPEMD-160 is used for Bitcoin addresses (160-bit). Ethereum uses Keccak-256 (last 20 bytes) instead. This exact flow is tested frequently as a step-by-step question.

QUICK REFERENCE: Key Terms and One-Line Definitions

Term	One-Line Definition
ECDLP	Given T=dP on elliptic curve, finding d is computationally infeasible - one-way function basis of ECC security
#E (Order of Elliptic Curve)	The total number of points in the cyclic group of the elliptic curve
Domain Parameters	Sextuple (p, a, b, G, n, h) fully defining an ECC curve and its group
SECP256K1	Standardized ECC curve used in Bitcoin/Ethereum: y^2 = x^3 + 7 over 256-bit prime field
p (in SECP256K1)	256-bit prime = 2^256 - 2^32 - 977; defines the finite field size
a, b (SECP256K1)	a=0, b=7; coefficients making the curve equation y^2 = x^3 + 7
G (Generator Point)	Base point that generates the cyclic subgroup; all public keys are multiples of G
n (Order)	Number of points in the cyclic subgroup generated by G; private key must be < n
h (Cofactor)	Ratio of total curve points to subgroup order; h=1 for secp256k1 (ideal for security)
Compressed Public Key	33 bytes: 02/03 prefix + x-coordinate; 02 if y even, 03 if y odd

Uncompressed Public Key	65 bytes: 04 prefix + x-coordinate (32 bytes) + y-coordinate (32 bytes)
PEM Format	Privacy Enhanced Mail; Base64-encoded cryptographic keys with header/footer lines
publicExponent	Common RSA public exponent value: 65537 = 2^16 + 1; Fermat prime enabling efficient computation
Digital Signature	Hash data then sign hash with private key; verify by decrypting signature with public key
Authenticity (DS property)	Digital signatures are verifiable by the receiving party
Unforgeability (DS property)	Only private key holder can produce a valid signature; same as non-repudiation
Non-Reusability (DS property)	A signature is bound to its specific message; cannot be reused for another message
Sign-then-Encrypt	Sign data with private key FIRST, then encrypt (data+signature) with receiver's public key - MORE SECURE
Encrypt-then-Sign	Encrypt data with receiver's public key FIRST, then sign the encrypted data - LESS SECURE
Certificate Authority (CA)	Trusted entity that issues digital certificates binding a public key to an identity
ECDSA	DSA applied to elliptic curves; based on ECDLP; used in Bitcoin and Ethereum for transaction signing
ECDSA Key Pair	Private key = integer d (0 < d < q); Public key = sextuple (p, a, b, q, A, B) where B = d*A
HASH160 (Bitcoin)	Combined hash operation: RIPEMD-160(SHA-256(public_key)); produces Bitcoin address core
Base58Check	Bitcoin's address encoding: Base58 alphabet (no 0, O, I, l) + 4-byte checksum for error detection
Bitcoin Address Derivation	Private key -> Q=d*G (ECC) -> SHA-256 -> RIPEMD-160 -> version+checksum -> Base58Check
Ethereum Address	Last 20 bytes of Keccak-256 hash of the public key; no RIPEMD-160, displayed as hex with 0x prefix

MUST-KNOW POINTS FOR EXAM

1. ECDLP: Given T = dP on elliptic curve, finding d is computationally infeasible. P (generator) and T (public key) are public; d (private key) is secret. Security relies on this one-way property.

2. ECC Domain Parameters sextuple: T = (p, a, b, G, n, h). p = field size, a/b = curve coefficients, G = generator/base point, n = order of G, h = cofactor.

3. SECP256K1 used in Bitcoin and Ethereum: y^2 = x^3 + 7 (a=0, b=7). p = 2^256 - 2^32 - 977 (256-bit prime). h=1 (cofactor). Private key: 0 < d < n.

4. Compressed public key = 33 bytes (02/03 + x only). Uncompressed = 65 bytes (04 + x + y). Compressed works because y can be recovered from x and the curve equation.

5. RSA private key components: modulus (n=p*q), publicExponent (65537), privateExponent (d), prime1 (p), prime2 (q).

6. RSA OpenSSL commands: genpkey -algorithm RSA (private key), rsa -pubout (derive public key), rsautl -encrypt -pubin (encrypt), rsautl -decrypt (decrypt).

7. ECC OpenSSL commands: ecparam -name secp256k1 -genkey (private key), ec -pubout (derive public key), ec -text -noout (view details), ecparam -list_curves (list all curves).

8. 3 Properties of Digital Signatures: (1) Authenticity - verifiable by receiver, (2) Unforgeability/Non-Repudiation - only private key holder can sign, (3) Non-Reusability - signature bound to specific message.

9. RSA Digital Signature: Step 1 = Hash the data (integrity). Step 2 = Sign hash with private key (authentication). Verification: Decrypt signature with public key, recompute hash, compare.

10. Sign-then-Encrypt (MORE SECURE): Sign with private key -> append signature -> encrypt (data+sig) with receiver's public key. Encrypt-then-Sign (LESS SECURE): Encrypt first, then sign encrypted data.

11. ECDSA key generation: Choose d randomly (0 < d < q). Compute B = d*A (public key). Public key = sextuple (p, a, b, q, A, B). Private key = integer d.

12. RSA key size vs ECC equivalence: 1024=160, 2048=224, 3072=256, 7680=384, 15360=521 (ECC bits). ECC 256-bit provides same security as RSA 3072-bit.

13. Bitcoin address derivation chain: Private key (256-bit) -> Public Key Q = d*G (secp256k1) -> SHA-256 -> RIPEMD-160 (160-bit) -> add version byte -> 4-byte checksum -> Base58Check encoding.

14. Ethereum uses Keccak-256 (last 20 bytes) for addresses - NOT RIPEMD-160. Ethereum addresses shown in hex with 0x prefix. Both Bitcoin and Ethereum use secp256k1 curve for ECDSA.